SAP Fiori Elements List Report Lab

Single CAP project: practise every UI annotation

Edition: WITH ANSWERS (tasks, then a worked solution for every story)

What you build

- **One project: "Service Desk Tickets"**, a CAP backend with draft, plus an SAP Fiori elements **List Report** (and a minimal Object Page).
- The focus is the **UI layer**: CDS/OData annotations (LineItem, SelectionFields, ValueList, SideEffects, variants and more) and the manifest settings that complement them.
- It ends with an **Excel upload** that creates table data, built twice: your own implementation and a ready-made component.
- Ten milestones, each a self-contained practice block with stories, detailed tasks, test scenarios and acceptance criteria.

How to learn with it

- Do the tasks in order. For each story first read the **topics to explore**, search the docs, then build.
- Write the **expected behaviour** before testing; the test scenarios list what to try.
- A story is done only when all **acceptance criteria** are true in the running app.
- Compare your result with the worked solution only after you tried (use the answers edition).
- Versions matter: SAPUI5, Fiori elements and CAP change often. Always confirm annotation names and manifest keys in $metadata, the vocabulary pages and the current documentation.

Prerequisites and setup

You need Node.js (a version supported by current CAP), the CDS development kit, VS Code with the SAP CDS language support, and ideally the SAP Fiori tools extensions (application generator, guided development, XML/CDS annotation language server). SAP Business Application Studio provides all of this in the browser.

No SAP system is required: the project runs on SQLite with CSV sample data. Basic knowledge of CAP (entities, services, running `cds watch`) is assumed; if you are new, work through the CAP getting-started guide first.

The data model you will implement (fields and purpose)

Fields of Tickets	Used for
ticketNumber, title, description	Identification and text; semantic key, search
category, subcategory (dependent), priority, status, channel	Code lists: value help, text, criticality, side effects
customer, assignee, assigneeTeam	Master data: standard value help, contact card, Out and Constant parameters
dueDate, createdAt	Date filters, sorting, SLA column
estimatedHours, hourlyRate, currency, cost (calculated)	Units, currency, side effect recalculation
completion, satisfaction	Progress and rating data points
isEscalated, attachmentUrl, resolution	Boolean filter/tab, URL column, action parameter
priorityCriticality, statusCriticality, isOpen, isClosed, fieldControl (calculated)	Technical fields that drive colours, enablement and field control
worklogs (composition)	Table on object page, side effect on entities

Supporting entities: Customers, Agents, Worklogs, and code lists Priorities, Statuses, Channels, Categories and Subcategories (dependent on Categories).

Annotation coverage matrix

Every row must be used by you at least once. Milestone 10 asks you to prove it.

Annotation / setting	Purpose	Milestone / story
UI.HeaderInfo, Common.SemanticKey, @title / Common.Label	Titles, key field, translatable labels	2.1
UI.LineItem: DataField, DataFieldWithUrl, DataFieldForAnnotation, DataFieldForAction	All column and button types	2.2-2.5, 5.2
UI.Importance, HTML5.CssDefaults, Common.QuickInfo	Responsive columns, widths, tooltips	2.2, 2.7
Criticality, CriticalityRepresentation, UI.DataPoint (Progress, Rating)	Colours, icons, progress bars, stars	2.3, 2.4
Communication.Contact, UI.IsImageURL, UI.FieldGroup	Contact card, images, combined column	2.5
Measures.ISOCurrency, Measures.Unit, Common.Text / TextArrangement, UI.Hidden	Formatting and hiding	2.3, 2.6
UI.SelectionFields, UI.FilterFacets, UI.HiddenFilter, Common.FilterDefaultValue	Filter bar design	3.1-3.3
Common.FilterExpressionType, Capabilities.FilterRestrictions / SortRestrictions	Filter and sort rules	3.4, 3.5
@cds.search, Search.defaultSearchElement	Search behaviour	3.5
@cds.odata.valuelist, Common.ValueList, ValueListWithFixedValues	Automatic, standard and dropdown value help	4.1-4.3
ValueListParameterInOut / In / Out / Constant / DisplayOnly	All parameter kinds, dependent value help	4.4, 4.5
Common.ValueListForValidation, qualified value lists	Validation and alternative value helps	4.6, 4.7

Annotation / setting	Purpose	Milestone / story
Core.OperationAvailable, Common.IsActionCritical, UI.ParameterDefaultValue, UI.MultiLineText	Dynamic and critical actions, parameter dialogs	5.3, 5.4
UI.DeleteHidden, UI.CreateHidden, Common.FieldControl, Capabilities.UpdateRestrictions	Data-driven UI state	5.6
Common.SideEffects: SourceProperties, TargetProperties, TargetEntities, actions	Keeping the page consistent	6.1-6.7
UI.SelectionVariant, UI.PresentationVariant, UI.SelectionPresentationVariant	Views (tabs), sort, group	7.2-7.4
UI.Chart, Aggregation.CustomAggregate, manifest views primary/secondary	Table and chart views	7.5
Manifest: tableSettings, initialLoad, variantManagement, custom column, custom action	Non-annotation configuration	7.1, 8.1-8.5, 9.4
Custom action + CAP action + spreadsheet parsing	Excel upload that creates table data	9.1-9.7

Roadmap

#	Milestone	Effort
1	Project Foundation (CAP backend + empty List Report)	4-6 hours
2	Columns and Field Display (UI.LineItem and friends)	8-10 hours
3	Filter Bar and Search	6-8 hours
4	Value Help (Common.ValueList in every flavour)	8-10 hours
5	Actions and Dynamic UI	8-10 hours
6	Side Effects	8-10 hours
7	Multiple Views, Variants and Presentation	8-12 hours
8	Table and Page Configuration (manifest.json)	6-8 hours
9	Excel Upload that Creates Table Data	10-14 hours
10	Review, Polish and Coverage Check	3-5 hours

Learning material (links)

Links were checked at the time of writing. If a page has moved, search for its title.

Official CAP documentation (capire)

- **CAP documentation home**: https://cap.cloud.sap/docs/
- **Serving Fiori UIs with CAP**: https://cap.cloud.sap/docs/advanced/fiori
- **OData in CAP (annotations, draft, actions)**: https://cap.cloud.sap/docs/advanced/odata

SAP Fiori elements (OData V4) and UI5

- **Side Effects (UI5 documentation topic)**: https://ui5.sap.com/sdk/docs/topics/18b17bdd49d1436fa9172cbb01e26544.html
- **Flexible Programming Model Explorer (live examples)**: https://ui5.sap.com/test-resources/sap/fe/core/fpmExplorer/index.html
- **SAP Fiori design guidelines (list report floorplan)**: https://experience.sap.com/fiori-design-web/
- **SAP Fiori tools documentation**: https://help.sap.com/docs/SAP_FIORI_tools
- **SAP Community: Fiori elements topic page**: https://community.sap.com/topics/fiori-elements

Sample projects to read (best learning material for annotations)

- **Fiori elements for OData V4 Feature Showcase (CAP)**: https://github.com/SAP-samples/fiori-elements-feature-showcase
- **ABAP Platform Fiori Feature Showcase (feature descriptions and code snippets)**: https://github.com/SAP-samples/abap-platform-fiori-feature-showcase
- **cap-sflight (travel app, annotations for actions, draft, value help)**: https://github.com/SAP-samples/cap-sflight
- **btp-cap-demo-usecases (list report table settings, actions via annotations)**: https://github.com/SAP-samples/btp-cap-demo-usecases

Annotation vocabularies (reference)

- **SAP OData vocabularies (UI, Common, Communication, Measures, Aggregation...)**: https://sap.github.io/odata-vocabularies/
- **Vocabularies source repository**: https://github.com/SAP/odata-vocabularies

Specific topics used in this lab

- **Multiple views on list report tables with CAP (blog)**: https://answers.sap.com/t5/technology-blogs-by-sap/cap-with-fiori-elements-configure-multiple-views-on-list-report-tables/ba-p/13579909
- **Mass upload from a spreadsheet into a Fiori app (blog)**: https://community.sap.com/t5/technology-blog-posts-by-sap/how-to-mass-upload-from-a-spreadsheet-file-into-an-sap-fiori-app-in-sap-btp/ba-p/13800792
- **UI5 Spreadsheet Importer documentation**: https://docs.spreadsheet-importer.com/
- **UI5 Spreadsheet Importer live demo**: https://livedemo.spreadsheet-importer.com/
- **UI5 Spreadsheet Importer source (GitHub)**: https://github.com/spreadsheetimporter/ui5-cc-spreadsheetimporter
- **Custom view/tab in a List Report with controller extension (blog)**: https://answers.sap.com/t5/technology-blogs-by-members/fiori-elements-add-geo-map-to-list-report-as-custom-view-odata-v4/ba-p/13551133

Study tip

The two feature showcases are the best companion: search them for the annotation name you practise (for example "FilterDefaultValue", "SideEffects", "SelectionPresentationVariant"), read the snippet, close it, and rebuild it in your own project.

Milestone 1 - Project Foundation (CAP backend + empty List Report)

Item	Detail
Mini project	Service Desk Tickets: a CAP project with a rich data model, sample data, a draft-enabled service and a generated List Report app, ready for annotations.
Level / effort	Beginner / 4-6 hours
Annotations practised	@odata.draft.enabled, @cds.odata.valuelist, calculated elements, annotation file layout, Fiori preview.

Learning outcomes

- Have a backend whose fields cover every annotation type you will practise.
- Run a generated List Report and know where annotations, manifest and i18n live.
- Know the development loop: edit annotation, save, refresh preview, inspect metadata.

Story 1.1 - Create the project and learn the loop

Goal: A fresh CAP project named desk-lab and a working development loop.

Your tasks

1. Create a CAP project named desk-lab, install dependencies, put it under Git.
2. Install the VS Code extensions: SAP CDS Language Support and, if available to you, SAP Fiori tools (application generator, guided development, XML annotation language server).
3. Read the CAP chapter "Serving Fiori UIs" and the first two pages of the Fiori elements documentation. Write in your journal what an annotation is and why Fiori elements needs no UI code.
4. Start the server in watch mode and open the welcome page. Find where the metadata ($metadata) of a service can be read.
5. Plan your folder layout: annotations will be split into several .cds files under app/tickets/annotations (columns, filters, value help, actions, side effects, variants). Create the empty folder.

Where to look / topics to explore

- capire: Serving Fiori UIs
- Fiori elements documentation: "Annotations" overview

Acceptance criteria

- The server runs and the welcome page opens.
- Your journal has your own definition of annotation, metadata, vocabulary.

Solution

Typical commands (run in a terminal; exact output differs by version):

```
terminal
cds init desk-lab
cd desk-lab
npm install
git init && git add . && git commit -m "init"
mkdir -p app/tickets/annotations
cds watch
```

Definition for your journal: an **annotation** is metadata attached to a model element that tells a generic framework how to present or treat it. **Vocabularies** (UI, Common, Communication, Capabilities, Measures) define which annotation terms exist. **$metadata** is the machine-readable service description in which the annotations are published.

Story 1.2 - Model the data (spec given, you write the CDS)

Goal: A data model rich enough for every list report feature.

Your tasks

1. Create entity **Tickets** (key UUID plus managed fields) with the fields from the table below. Decide the CDS types yourself.
2. Create the code-list entities Priorities, Statuses, Channels, Categories and the dependent code list Subcategories (belongs to a category). Use the standard CAP code-list aspect.
3. Create Customers, Agents and Worklogs (a composition of many on Tickets).
4. Add **calculated elements** to Tickets: cost (estimated hours times hourly rate), priorityCriticality (High=1 negative, Medium=2 critical, Low=3 positive, otherwise 0), statusCriticality (New=2, In progress=2, Waiting=0, Closed=3), isClosed, isOpen and fieldControl (1 = read-only when closed, otherwise 3).
5. Compile the model to SQL and check that all foreign keys and calculated columns exist. Explain in your journal why we model priority and status as associations, not strings.

Where to look / topics to explore

- Calculated elements and the case expression in CDS
- sap.common.CodeList aspect
- Managed associations and generated foreign key names

Acceptance criteria

- The model compiles without errors.
- Foreign keys are named like priority_code, customer_ID.
- You can explain what a calculated element is and when it is evaluated.

Solution

Full model:

db/schema.cds

```
namespace desk;
using { cuid, managed, Currency, sap.common.CodeList } from '@sap/cds/common';

entity Tickets : cuid, managed {
  ticketNumber   : String(10);
  title          : String(120);
  description    : String(1000);
  category       : Association to Categories;
  subcategory    : Association to Subcategories;
  priority       : Association to Priorities;
  status         : Association to Statuses;
  channel        : Association to Channels;
  customer       : Association to Customers;
  assignee       : Association to Agents;
  assigneeTeam   : String(40);
  dueDate        : Date;
  estimatedHours : Decimal(5,1);
  hourlyRate     : Decimal(9,2);
  currency       : Currency;
  completion     : Integer;
  satisfaction   : Integer;
  isEscalated    : Boolean default false;
  attachmentUrl  : String(500);
  resolution     : String(200);
  worklogs       : Composition of many Worklogs on worklogs.ticket = $self;

  // ---- calculated elements (evaluated on read) ----
  cost : Decimal(12,2) = estimatedHours * hourlyRate;
  priorityCriticality : Integer = case when priority.code = 'H' then 1
                                       when priority.code = 'M' then 2
                                       when priority.code = 'L' then 3
                                       else 0 end;
  statusCriticality   : Integer = case when status.code = 'N' then 2
                                       when status.code = 'I' then 2
                                       when status.code = 'C' then 3
                                       else 0 end;
  isClosed     : Boolean = case when status.code = 'C' then true else false end;
```

```
  isOpen       : Boolean = case when status.code = 'C' then false else true end;
  fieldControl : Integer = case when status.code = 'C' then 1 else 3 end;
}

entity Worklogs : cuid {
  ticket   : Association to Tickets;
  loggedBy : Association to Agents;
  hours    : Decimal(4,1);
  note     : String(200);
}

entity Priorities    : CodeList { key code : String(1);  }
entity Statuses      : CodeList { key code : String(1);  }
entity Channels      : CodeList { key code : String(10); }
entity Categories    : CodeList { key code : String(10); }
entity Subcategories : CodeList { key code : String(20); category : Association to Categories; }

entity Customers : cuid {
  name : String(80); email : String(120); phone : String(30);
  country : String(2); avatarUrl : String(300);
}
entity Agents : cuid { name : String(80); email : String(120); team : String(40); }
```

If your CAP version rejects a calculated element (for example the case expression or the path through the association key), compute the field in an after READ handler and declare it as `virtual` in the service. Note in your journal which way you went.

Why associations: code lists give you translatable texts, value help, referential integrity and a single place for criticality logic.

Story 1.3 - Create sample data

Goal: Enough realistic rows to see sorting, grouping, criticality and tabs.

Your tasks

1. Create CSV files for the code lists: 3 priorities (H, M, L), 4 statuses (N New, I In Progress, W Waiting, C Closed), 4 channels (EMAIL, PHONE, WEB, CHAT), 4 categories (HW, SW, NET, ACC) and 8 subcategories (two per category).
2. Create 4 customers (with e-mail, phone, country and an avatar URL) and 4 agents (team Support or Network).
3. Create at least 30 tickets covering: all statuses, all priorities, past and future due dates, some escalated, completion between 0 and 100, satisfaction between 1 and 5 for closed tickets, different currencies (EUR, USD, INR), some without an assignee.
4. Restart and check the data through the service in the browser.

Where to look / topics to explore

- CSV file naming rule: namespace-Entity.csv
- Use a spreadsheet to generate rows, then export CSV

Acceptance criteria

- 30+ tickets are visible and cover every status and priority.
- Every foreign key in the CSV points to an existing row.

Solution

Code lists (one file each, in db/data):

```
one CSV per code list
# desk-Priorities.csv
code,name,descr
H,High,Needs immediate action
M,Medium,Normal handling
L,Low,Can wait

# desk-Statuses.csv
code,name,descr
N,New,
I,In Progress,
```

```
W,Waiting,Waiting for customer
C,Closed,

# desk-Channels.csv
code,name,descr
EMAIL,E-Mail,
PHONE,Phone,
WEB,Web portal,
CHAT,Chat,

# desk-Categories.csv
code,name,descr
HW,Hardware,
SW,Software,
NET,Network,
ACC,Access,

# desk-Subcategories.csv
code,name,descr,category_code
HW-LAP,Laptop,,HW
HW-PRT,Printer,,HW
SW-INST,Installation,,SW
SW-BUG,Defect,,SW
NET-VPN,VPN,,NET
NET-WIFI,Wi-Fi,,NET
ACC-PWD,Password reset,,ACC
ACC-NEW,New account,,ACC
```

Master data and the first tickets (extend to 30+ rows with a spreadsheet; keep the UUID pattern valid):

```
sample rows
# desk-Customers.csv
ID,name,email,phone,country,avatarUrl
c0000000-0000-0000-0000-000000000001,Anna Becker,anna.becker@example.com,+49 30 111111,DE,https://i.pravatar.cc/
    80?img=5
c0000000-0000-0000-0000-000000000002,Ravi Menon,ravi.menon@example.com,+91 80 222222,IN,https://i.pravatar.cc/80
    ?img=12
c0000000-0000-0000-0000-000000000003,Lucy Grant,lucy.grant@example.com,+1 212 333333,US,https://i.pravatar.cc/80
    ?img=32
c0000000-0000-0000-0000-000000000004,Omar Haddad,omar.haddad@example.com,+971 4 444444,AE,https://i.pravatar.cc/
    80?img=15

# desk-Agents.csv
ID,name,email,team
a0000000-0000-0000-0000-000000000001,Mia Cooper,mia.cooper@example.com,Support
a0000000-0000-0000-0000-000000000002,Jon Park,jon.park@example.com,Support
a0000000-0000-0000-0000-000000000003,Sara Lund,sara.lund@example.com,Network
a0000000-0000-0000-0000-000000000004,Dev Patel,dev.patel@example.com,Network

# desk-Tickets.csv (first rows)
ID,ticketNumber,title,description,category_code,subcategory_code,priority_code,status_code,channel_code,customer
    _ID,assignee_ID,assigneeTeam,dueDate,estimatedHours,hourlyRate,currency_code,completion,satisfaction,isEscal
    ated,attachmentUrl,resolution
t0000000-0000-0000-0000-000000000001,T-00001,Laptop does not boot,Black screen after update,HW,HW-LAP,H,I,PHONE,
    c0000000-0000-0000-0000-000000000001,a0000000-0000-0000-0000-000000000001,Support,2026-11-12,3.5,60,EUR,40,,
    true,https://example.com/doc/1,
t0000000-0000-0000-0000-000000000002,T-00002,VPN drops every hour,Disconnects during calls,NET,NET-VPN,M,N,EMAIL
    ,c0000000-0000-0000-0000-000000000002,a0000000-0000-0000-0000-000000000003,Network,2026-11-20,2,55,USD,0,,fa
    lse,,
t0000000-0000-0000-0000-000000000003,T-00003,Reset password,User locked out,ACC,ACC-PWD,L,C,WEB,c0000000-0000-00
    00-0000-000000000003,a0000000-0000-0000-0000-000000000002,Support,2026-10-01,0.5,40,USD,100,5,false,,Passwor
    d reset done
```

Tip: generate the remaining rows in a spreadsheet using formulas for dates (some before 2026-11-05 to create overdue tickets), then export to CSV. Keep at least two tickets without assignee_ID.

Story 1.4 - Expose the service with draft and value-list entities

Goal: A service the List Report can consume.

Your tasks

1. Create the service DeskService with path desk.
2. Expose Tickets as a draft-enabled projection. Expose Customers, Agents, Priorities, Statuses, Channels, Categories, Subcategories and Worklogs.

3. Make the code lists read-only and mark Channels and Statuses with the annotation that lets CAP generate a value help automatically.
4. Open $metadata and find: the entity set Tickets, the draft properties (IsActiveEntity, HasDraftEntity), the calculated fields and the generated value list annotations.

Where to look / topics to explore

- @odata.draft.enabled
- @cds.odata.valuelist
- @readonly

Acceptance criteria

- Tickets can be read with expanded customer and priority.
- You can point to the draft properties in $metadata.

Solution

Service definition (actions are added in Milestone 5, the upload action in Milestone 9):

srv/service.cds

```
using { desk } from '../db/schema';

@path: 'desk'
service DeskService {

  @odata.draft.enabled
  entity Tickets as projection on desk.Tickets;

  entity Worklogs  as projection on desk.Worklogs;
  entity Customers as projection on desk.Customers;
  entity Agents    as projection on desk.Agents;

  @readonly entity Priorities    as projection on desk.Priorities;
  @readonly entity Categories    as projection on desk.Categories;
  @readonly entity Subcategories as projection on desk.Subcategories;
  @readonly @cds.odata.valuelist entity Statuses as projection on desk.Statuses;
  @readonly @cds.odata.valuelist entity Channels as projection on desk.Channels;
}
```

In `$metadata` look for the properties `IsActiveEntity`, `HasActiveEntity`, `HasDraftEntity` and for the annotation `Common.ValueList` generated for `channel_code` and `status_code`.

Story 1.5 - Generate the List Report app and prepare the workspace

Goal: A runnable List Report with a manifest, i18n and separate annotation files.

Your tasks

1. Use the Fiori application generator: template List Report Page, OData V4, data source local CAP project, main entity Tickets, object page enabled. Application id: desk.tickets.
2. Open the generated manifest.json. Locate: the data source, the model, the routing targets (list and object page), the sap.fe.templates.ListReport settings.
3. Move annotation work to app/tickets/annotations/*.cds files. Create empty files for columns, filters, valuehelp, actions, sideeffects, variants and import them from one annotations.cds using the CDS using statement.
4. Create i18n: decide where i18n.properties lives and how annotations refer to it with {i18n>key}. Add keys ticket and tickets.
5. Run the app and check the preview shows a table without columns (this is expected).

Where to look / topics to explore

- Fiori tools: Application Generator
- manifest.json: sap.ui5.routing.targets
- capire: i18n in annotations

Acceptance criteria

- The app opens from the welcome page.
- You can show where each annotation file is loaded.

Solution

Annotation entry file that imports the topic files:

app/tickets/annotations.cds

```
using from './annotations/columns';
using from './annotations/filters';
using from './annotations/valuehelp';
using from './annotations/actions';
using from './annotations/sideeffects';
using from './annotations/variants';
```

Every topic file starts with the same line:

app/tickets/annotations/columns.cds (and the others)

```
using DeskService as service from '../../../srv/service';
```

Parts of the generated manifest you should recognise:

webapp/manifest.json (excerpt)

```
"sap.app": { "id": "desk.tickets", "dataSources": { "mainService": { "uri": "/odata/v4/desk/", "type": "OData",
             "settings": { "odataVersion": "4.0" } } } },
"sap.ui5": { "routing": { "targets": {
   "TicketsList": { "type": "Component", "id": "TicketsList", "name": "sap.fe.templates.ListReport",
                    "options": { "settings": { "contextPath": "/Tickets", "variantManagement": "Control",
                                  "navigation": { "Tickets": { "detail": { "route": "TicketsObjectPage" } } } } }
     },
   "TicketsObjectPage": { "type": "Component", "id": "TicketsObjectPage", "name": "sap.fe.templates.ObjectPage",
                    "options": { "settings": { "contextPath": "/Tickets", "editableHeaderContent": false } } } }
     } }
```

i18n: with CAP the file `app/tickets/i18n/i18n.properties` (or the project `_i18n`) is picked up; annotations refer to keys with `{i18n>key}`. Example keys: `ticket=Ticket`, `tickets=Tickets`.

Milestone checkpoint

Question	Answer
Why does Fiori elements need annotations?	It generates the UI at runtime from service metadata; annotations tell it which fields are columns, filters, labels, actions.
Why are priority and status associations to code lists?	Consistency, texts/translations, value help generation, and criticality can be derived from the key.

Milestone 2 - Columns and Field Display (UI.LineItem and friends)

Item	Detail
Mini project	A complete list report table: every kind of column the framework supports.
Level / effort	Intermediate / 8-10 hours
Annotations practised	UI.HeaderInfo, Common.SemanticKey, UI.LineItem (DataField, DataFieldWithUrl, DataFieldForAnnotation, DataFieldForAction), UI.Importance, Criticality, UI.DataPoint (Progress, Rating), Communication.Contact, Measures.ISOCurrency/Unit, Common.Text and TextArrangement, UI.Hidden, HTML5.CssDefaults, UI.FieldGroup.

Learning outcomes

- Build a table from scratch using only annotations.
- Choose the correct column type for each kind of data.
- Control responsive behaviour with importance and widths.

Story 2.1 - Header info, semantic key and labels

Goal: Give the entity a name, a title field and translatable labels.

Your tasks

1. Annotate Tickets with HeaderInfo: type name, plural type name (from i18n), title = title field, description = ticket number.
2. Declare the ticket number as the semantic key (it is shown in bold with draft state indication).
3. Add translatable labels for all fields via i18n and the label annotation (decide whether you use Common.Label or the title annotation, and explain the difference).
4. Check the browser tab title and the empty list message use the type name.

Where to look / topics to explore

- UI.HeaderInfo
- Common.SemanticKey
- Common.Label versus @title

Acceptance criteria

- Labels come from i18n.
- The semantic key is shown prominently in the table.

Solution

Header info, semantic key and labels:

app/tickets/annotations/columns.cds

```
annotate service.Tickets with @(
  Common.SemanticKey : [ticketNumber],
  UI.HeaderInfo : {
    TypeName       : '{i18n>ticket}',
    TypeNamePlural : '{i18n>tickets}',
    Title          : { $Type : 'UI.DataField', Value : title },
    Description    : { $Type : 'UI.DataField', Value : ticketNumber }
  }
);

annotate service.Tickets with {
  ticketNumber   @title : '{i18n>ticketNumber}';
  title          @title : '{i18n>title}';
  priority       @title : '{i18n>priority}';
  status         @title : '{i18n>status}';
  customer       @title : '{i18n>customer}';
  assignee       @title : '{i18n>assignee}';
```

```
  dueDate        @title : '{i18n>dueDate}';
  estimatedHours @title : '{i18n>estimatedHours}';
  cost           @title : '{i18n>cost}';
};
```

Difference: `@title` is the CAP shortcut that becomes Common.Label in the metadata; `Common.Label` is the vocabulary term itself. Use one style consistently.

Story 2.2 - First columns with importance and widths

Goal: A table with plain DataField columns that behaves responsively.

Your tasks

1. Define a LineItem with: ticket number, title, priority, status, customer, assignee, due date, cost, estimated hours.
2. Assign importance High, Medium or Low to each column. Resize the browser to phone width and observe which columns disappear. Adjust until a phone shows ticket number, title and status.
3. Set explicit widths for ticket number (8rem) and title (20rem) with the HTML5 CSS defaults annotation.
4. Reorder the columns in the annotation and observe the table.

Where to look / topics to explore

- UI.Importance on DataField
- HTML5.CssDefaults

Test scenarios (do them all)

- Browser width 1400px, 800px, 400px: write which columns you see at each width.

Acceptance criteria

- Phone width shows exactly three columns.
- Ticket number and title have the specified widths.

Solution

Plain columns with importance:

app/tickets/annotations/columns.cds

```
annotate service.Tickets with @(
  UI.LineItem : [
    { $Type : 'UI.DataField', Value : ticketNumber,    ![@UI.Importance] : #High },
    { $Type : 'UI.DataField', Value : title,           ![@UI.Importance] : #High },
    { $Type : 'UI.DataField', Value : priority_code,   ![@UI.Importance] : #Medium },
    { $Type : 'UI.DataField', Value : status_code,     ![@UI.Importance] : #High },
    { $Type : 'UI.DataField', Value : customer_ID,     ![@UI.Importance] : #Medium },
    { $Type : 'UI.DataField', Value : assignee_ID,     ![@UI.Importance] : #Medium },
    { $Type : 'UI.DataField', Value : dueDate,         ![@UI.Importance] : #Medium },
    { $Type : 'UI.DataField', Value : cost,            ![@UI.Importance] : #Low },
    { $Type : 'UI.DataField', Value : estimatedHours,  ![@UI.Importance] : #Low }
  ]
);

annotate service.Tickets with {
  ticketNumber @HTML5.CssDefaults : { width : '8rem' };
  title        @HTML5.CssDefaults : { width : '20rem' };
};
```

Rule: High importance columns stay on small screens, Low importance columns are hidden first.

Story 2.3 - Criticality colours and text arrangement

Goal: Status and priority look meaningful: colour, icon and text instead of codes.

Your tasks

1. Add criticality to priority and status columns using your calculated criticality fields. Show an icon for priority.
2. Show text instead of code for priority, status, customer, assignee, category and subcategory using the Common.Text annotation. Decide per field whether to use text only, text first or text last and justify.

3. Hide the technical criticality fields from table personalisation and filter.

Where to look / topics to explore

- Criticality and CriticalityRepresentation on UI.DataField
- Common.Text, Common.TextArrangement (#TextOnly, #TextFirst, #TextLast, #TextSeparate)
- UI.Hidden

Test scenarios (do them all)

- High priority shows red with icon, Low shows green.
- Closed ticket status is green, New is orange.

Acceptance criteria

- No technical code or UUID is visible in the table.
- Colours match your mapping.

Solution

Criticality, icon and texts:

app/tickets/annotations/columns.cds

```
annotate service.Tickets with @(
  UI.LineItem : [
    { $Type : 'UI.DataField', Value : ticketNumber, ![@UI.Importance] : #High },
    { $Type : 'UI.DataField', Value : title,        ![@UI.Importance] : #High },
    { $Type : 'UI.DataField', Value : priority_code,
      Criticality : priorityCriticality, CriticalityRepresentation : #WithIcon,
      ![@UI.Importance] : #Medium },
    { $Type : 'UI.DataField', Value : status_code,
      Criticality : statusCriticality, ![@UI.Importance] : #High }
    // ... remaining columns as before
  ]
);

annotate service.Tickets with {
  priority    @Common.Text : priority.name    @Common.TextArrangement : #TextOnly;
  status      @Common.Text : status.name      @Common.TextArrangement : #TextOnly;
  customer    @Common.Text : customer.name    @Common.TextArrangement : #TextOnly;
  assignee    @Common.Text : assignee.name    @Common.TextArrangement : #TextOnly;
  category    @Common.Text : category.name    @Common.TextArrangement : #TextOnly;
  subcategory @Common.Text : subcategory.name @Common.TextArrangement : #TextOnly;
};

annotate service.Tickets with {
  priorityCriticality @UI.Hidden @UI.HiddenFilter;
  statusCriticality   @UI.Hidden @UI.HiddenFilter;
  fieldControl        @UI.Hidden @UI.HiddenFilter;
  isOpen              @UI.Hidden @UI.HiddenFilter;
  isClosed            @UI.Hidden @UI.HiddenFilter;
};
```

Text arrangement guide: #TextOnly for small code lists, #TextFirst for master data where the ID helps (for example customer name first, ID in brackets), #TextLast when the code is more important.

Story 2.4 - Progress and rating data points

Goal: Show completion as a progress bar and satisfaction as stars.

Your tasks

1. Define a DataPoint for completion with visualization Progress and target value 100.
2. Define a DataPoint for satisfaction with visualization Rating and target value 5.
3. Add both to the table through DataFieldForAnnotation columns with labels.
4. Add a DataPoint for cost that shows the number in the colour of the ticket priority (research which DataPoint properties control colour).
5. Test with a ticket that has completion 0, 55 and 100 and with a missing rating.

Where to look / topics to explore

- UI.DataPoint, Visualization enum
- UI.DataFieldForAnnotation

Acceptance criteria

- Progress bars show the right fill.
- Open tickets show no stars without errors.

Solution

Data points and the columns that show them:

app/tickets/annotations/columns.cds

```
annotate service.Tickets with @(
  UI.DataPoint #Completion : {
    Value : completion, Title : 'Completion', Visualization : #Progress, TargetValue : 100
  },
  UI.DataPoint #Satisfaction : {
    Value : satisfaction, Title : 'Satisfaction', Visualization : #Rating, TargetValue : 5
  },
  UI.DataPoint #Cost : {
    Value : cost, Title : 'Cost', Criticality : priorityCriticality
  }
);

// inside the LineItem array:
//  { $Type : 'UI.DataFieldForAnnotation', Target : '@UI.DataPoint#Completion',   Label : 'Progress' },
//  { $Type : 'UI.DataFieldForAnnotation', Target : '@UI.DataPoint#Satisfaction', Label : 'Rating' },
```

ObjectNumber is not a visualization in the Visualization enum; a plain DataPoint with a Criticality renders the number in colour, which is the common alternative.

Story 2.5 - Contact card, link, image and combined column

Goal: Columns that contain richer content.

Your tasks

1. Annotate Customers with a contact card (full name, e-mail, phone) and show it as a column in Tickets using the navigation path. Click it and verify the popover.
2. Add a column with a URL (attachmentUrl) using the URL data field type and make the label a fixed text.
3. Show a customer avatar: research the annotation to mark a property as image URL and the HeaderInfo ImageUrl.
4. Create a combined column "Ownership" showing assignee and team in one cell using a FieldGroup and DataFieldForAnnotation.

Where to look / topics to explore

- Communication.Contact
- UI.DataFieldWithUrl
- UI.IsImageURL
- UI.FieldGroup

Acceptance criteria

- The contact popover shows e-mail and phone.
- The link opens in a new tab.
- The combined column shows two values.

Solution

Contact card, link, image and combined column:

app/tickets/annotations/columns.cds

```
annotate service.Customers with @(
  Communication.Contact : {
```

```
    fn    : name,
    email : [{ type : #work, address : email }],
    tel   : [{ type : #work, uri : phone }]
  }
);
annotate service.Customers with { avatarUrl @UI.IsImageURL; };

annotate service.Tickets with @(
  UI.FieldGroup #Ownership : { Data : [
    { $Type : 'UI.DataField', Value : assignee_ID },
    { $Type : 'UI.DataField', Value : assigneeTeam }
  ]},
  UI.LineItem : [
    ...,
    { $Type : 'UI.DataFieldForAnnotation', Target : 'customer/@Communication.Contact', Label : 'Customer' },
    { $Type : 'UI.DataFieldWithUrl', Value : attachmentUrl, Url : attachmentUrl, Label : 'Document' },
    { $Type : 'UI.DataFieldForAnnotation', Target : '@UI.FieldGroup#Ownership', Label : 'Ownership' }
  ]
);
```

The ellipsis `...` inside an array annotation means "keep the entries defined elsewhere", so you can extend a LineItem from several files (it needs a recent CDS compiler; otherwise keep one LineItem array). HeaderInfo `ImageUrl` can point to a property to show an image on the object page.

Story 2.6 - Currency, units and number formatting

Goal: Numbers are formatted professionally.

Your tasks

1. Mark cost and hourly rate as amounts in the currency of the row.
2. Mark estimated hours with the unit h.
3. Check how rows with EUR, USD and INR format differently and how the column header and sorting behave.
4. Find what happens if the currency field is not part of the table and explain.

Where to look / topics to explore

- Measures.ISOCurrency
- Measures.Unit

Acceptance criteria

- Currencies render with the correct decimals and symbols.

Solution

Currency and unit:

app/tickets/annotations/columns.cds

```
annotate service.Tickets with {
  cost          @Measures.ISOCurrency : currency_code;
  hourlyRate    @Measures.ISOCurrency : currency_code;
  estimatedHours @Measures.Unit       : 'h';
};
```

If the currency property is not a column, Fiori elements still formats the amount because it requests the currency property in the same read; sorting a mixed-currency column sorts by number only.

Story 2.7 - Table-level look: sorting default and column tooltips (stretch)

Goal: Polish.

Your tasks

1. Define a default sort order using a presentation variant (research; fully used in Milestone 7).
2. Add QuickInfo (tooltip) to three columns.
3. Make the escalated flag appear as an icon column with criticality instead of true/false.

Where to look / topics to explore

- Common.QuickInfo
- Criticality on Boolean via calculated criticality

Acceptance criteria

- Tooltips show; escalated tickets have a red marker.

Solution

Tooltips and default sort (the sort is activated as part of a presentation variant in Milestone 7):

app/tickets/annotations/columns.cds

```
annotate service.Tickets with {
  cost           @Common.QuickInfo : 'Estimated hours x hourly rate';
  completion     @Common.QuickInfo : 'Percent of work done';
  satisfaction   @Common.QuickInfo : 'Customer rating 1 to 5';
};

annotate service.Tickets with @(
  UI.PresentationVariant : {
    SortOrder : [{ Property : dueDate, Descending : false }],
    Visualizations : ['@UI.LineItem']
  }
);
```

For an escalated icon column, expose a small integer criticality field (1 when escalated, otherwise 0) next to the Boolean and use it in `Criticality`.

Milestone checkpoint

Question	Answer
Difference between DataField and DataFieldForAnnotation?	DataField shows one property; DataFieldForAnnotation shows another annotation (DataPoint, Contact, FieldGroup, Chart) in the cell.
What do the criticality values 0-3 mean?	0 neutral, 1 negative (red), 2 critical (orange), 3 positive (green).
How does importance drive responsiveness?	Low importance columns are hidden first on small screens (responsive table popin).

Milestone 3 - Filter Bar and Search

Item	Detail
Mini project	A filter bar with groups, defaults, restrictions and tuned search.
Level / effort	Intermediate / 6-8 hours
Annotations practised	UI.SelectionFields, UI.FilterFacets, Common.FilterDefaultValue, UI.HiddenFilter, Common.FilterExpressionType, Capabilities.FilterRestrictions / SortRestrictions, @cds.search, Search.defaultSearchElement.

Learning outcomes

- Design a usable filter bar.
- Restrict what users may filter and sort.
- Control how the search box behaves.

Story 3.1 - Selection fields

Goal: The visible filters of the page.

Your tasks

1. Define selection fields: ticket number, status, priority, category, assignee, customer, due date, escalated.
2. Observe which control each field gets (dropdown, date range, checkbox, value help input).
3. Use the Adapt Filters dialog to add title, channel and satisfaction filters and save them as a variant.

Where to look / topics to explore

- UI.SelectionFields

Acceptance criteria

- The filter bar shows eight filters.
- A saved variant restores them.

Solution

Selection fields:

```
app/tickets/annotations/filters.cds
annotate service.Tickets with @(
  UI.SelectionFields : [
    ticketNumber, status_code, priority_code, category_code,
    assignee_ID, customer_ID, dueDate, isEscalated
  ]
);
```

Story 3.2 - Filter groups

Goal: Structured Adapt Filters dialog.

Your tasks

1. Define filter facets (groups): General (status, priority, category), People (assignee, customer), Dates (due date, created at), Metrics (cost, estimated hours, completion).
2. Open Adapt Filters and verify the groups.
3. Write what happens to fields that are not in any group.

Where to look / topics to explore

- UI.FilterFacets
- UI.FieldGroup with qualifier

Acceptance criteria

- Four groups appear in the dialog.

Solution

Filter groups:

app/tickets/annotations/filters.cds

```
annotate service.Tickets with @(
  UI.FieldGroup #FGGeneral : { Data : [
    { Value : status_code }, { Value : priority_code }, { Value : category_code } ] },
  UI.FieldGroup #FGPeople  : { Data : [ { Value : assignee_ID }, { Value : customer_ID } ] },
  UI.FieldGroup #FGDates   : { Data : [ { Value : dueDate }, { Value : createdAt } ] },
  UI.FieldGroup #FGMetrics : { Data : [ { Value : cost }, { Value : estimatedHours }, { Value : completion } ] }
    ,
  UI.FilterFacets : [
    { $Type : 'UI.ReferenceFacet', Label : 'General', Target : '@UI.FieldGroup#FGGeneral' },
    { $Type : 'UI.ReferenceFacet', Label : 'People',  Target : '@UI.FieldGroup#FGPeople'  },
    { $Type : 'UI.ReferenceFacet', Label : 'Dates',   Target : '@UI.FieldGroup#FGDates'   },
    { $Type : 'UI.ReferenceFacet', Label : 'Metrics', Target : '@UI.FieldGroup#FGMetrics' }
  ]
);
```

Fields that are filterable but in no group are listed in a default group in the dialog.

Story 3.3 - Default values and hidden filters

Goal: Opinionated defaults.

Your tasks

1. Default the status filter to a sensible value. Observe that the page loads filtered; decide whether that is good and note the risk (users think data is missing).
2. Hide cost and hourly rate from the filter dialog while keeping them as columns.
3. Check the manifest option that controls initial loading and test with filter bar required.

Where to look / topics to explore

- Common.FilterDefaultValue
- UI.HiddenFilter
- manifest initialLoad

Acceptance criteria

- Defaults apply on first load.
- Cost is not filterable in the UI.

Solution

Defaults and hidden filters:

app/tickets/annotations/filters.cds

```
annotate service.Tickets with {
  status_code @Common.FilterDefaultValue : 'N';
  cost        @UI.HiddenFilter;
  hourlyRate  @UI.HiddenFilter;
};
```

Manifest: control the first load of data (values: Enabled, Disabled, Auto):

manifest.json

```
"sap.fe.templates.ListReport": { "settings": { "initialLoad": "Enabled" } }
```

Risk: a default filter hides data; show the active filters clearly (the filter bar does) and teach users. Prefer variants for personal defaults.

Story 3.4 - Filter expression types and restrictions

Goal: Control how users can enter filter values.

Your tasks

1. Make due date a single range filter, status a multi-value filter, ticket number a search-expression field and priority single value.
2. Declare title and description as not filterable on the server side. Test a manual request with a filter on description.
3. Make the status filter mandatory (required property). Observe the Go button and the asterisk.
4. Research and describe the difference between the UI annotation and the Capabilities restriction.

Where to look / topics to explore

- Common.FilterExpressionType
- Capabilities.FilterRestrictions: RequiredProperties, NonFilterableProperties, FilterExpressionRestrictions

Test scenarios (do them all)

- Leave required status empty and press Go.
- Enter two ranges in a single range field.

Acceptance criteria

- Each filter behaves as specified.
- Restrictions appear in $metadata.

Solution

Expression types (check the vocabulary for the exact allowed values in your UI5 version):

app/tickets/annotations/filters.cds

```
annotate service.Tickets with {
  dueDate      @Common.FilterExpressionType : 'SingleRange';
  status_code  @Common.FilterExpressionType : 'MultiValue';
  ticketNumber @Common.FilterExpressionType : 'SearchExpression';
  priority_code @Common.FilterExpressionType : 'SingleValue';
};

annotate service.Tickets with @(
  Capabilities.FilterRestrictions : {
    RequiredProperties      : [ status_code ],
    NonFilterableProperties : [ title, description ]
  }
);
```

UI annotations shape the filter control; Capabilities restrictions are part of the service contract and are honoured by Fiori elements and other clients. Neither replaces server-side authorisation.

Story 3.5 - Search and sort behaviour

Goal: The search box finds what users expect.

Your tasks

1. Search by title, description, ticket number and customer name. Configure which fields are searched.
2. Make the ticket number and title default search elements.
3. Disallow sorting by description.
4. Test fuzzy versus plain search and write the limits of SQLite search.

Where to look / topics to explore

- @cds.search
- Search.defaultSearchElement
- Capabilities.SortRestrictions

Acceptance criteria

- Searching a customer name finds the tickets.
- Description sorting is not offered.

Solution

Search and sort:

app/tickets/annotations/filters.cds

```
annotate service.Tickets with @cds.search : { title, description, ticketNumber, customer.name };

annotate service.Tickets with {
  ticketNumber @Search.defaultSearchElement;
  title        @Search.defaultSearchElement;
};

annotate service.Tickets with @(
  Capabilities.SortRestrictions : { NonSortableProperties : [ description ] }
);
```

SQLite does plain LIKE search; fuzzy search needs HANA features and further annotations (Search.fuzzinessThreshold).

Milestone checkpoint

Question	Answer
Difference between SelectionFields and FilterFacets?	SelectionFields = filters visible in the bar; FilterFacets = grouping in Adapt Filters.
Why also restrict on the server?	UI hiding is not security; direct API calls can bypass the UI.

Milestone 4 - Value Help (Common.ValueList in every flavour)

Item	Detail
Mini project	Every field that points to other data gets the right kind of value help, in the filter bar and in edit mode.
Level / effort	Intermediate to Advanced / 8-10 hours
Annotations practised	Common.ValueList, ValueListWithFixedValues, ValueListParameterInOut / In / Out / DisplayOnly / Constant, @cds.odata.valuelist, Common.Text with value help, dependent value help.

Learning outcomes

- Choose between dropdown, standard value help and automatic value help.
- Pass values between field and value help in all directions.
- Build a dependent (cascading) value help.

Story 4.1 - Automatic value help

Goal: Understand what CAP generates for you.

Your tasks

1. Open the Object Page of a ticket in edit mode and the channel and status fields. Find where the value help comes from (the annotation of the target entity).
2. In $metadata locate the generated Common.ValueList on channel_code. Write down which parameters it contains.
3. Remove the automatic annotation from Channels, observe the result, restore it.

Where to look / topics to explore

- @cds.odata.valuelist and @cds.odata.valuelist.exclude

Acceptance criteria

- You can explain what the generated annotation contains and its limits (no custom columns, no filters).

Solution

What CAP generates for an entity marked `@cds.odata.valuelist`: a `Common.ValueList` on every association that targets it, with `CollectionPath` = entity set, one InOut parameter for the key and display parameters for the first text-like columns. Limits: you cannot choose the columns, add filters (In/Constant) or Out parameters, nor make it a dropdown. That is why you write explicit annotations for customers, agents and subcategories.

Excerpt of what you should find in $metadata (shape only):

```
metadata (shape)
 <Annotation Term="Common.ValueList">
   <Record Type="Common.ValueListType">
     <PropertyValue Property="CollectionPath" String="Channels"/>
     <PropertyValue Property="Parameters"> ... ValueListParameterInOut(channel_code -> code), ...DisplayOnly(name
     ) </PropertyValue>
   </Record>
 </Annotation>
```

Story 4.2 - Dropdown (fixed values) for priority

Goal: A small code list becomes a dropdown instead of a dialog.

Your tasks

1. Write an explicit ValueList for priority_code that points to the Priorities collection and returns the code.
2. Mark it as fixed-values so a dropdown is shown. Display the name through the text annotation.
3. Check filter bar and edit mode: both should show a dropdown.

4. Write when a dropdown is appropriate (limit on the number of entries).

Where to look / topics to explore

- Common.ValueListWithFixedValues
- Common.ValueListParameterInOut
- Common.ValueListParameterDisplayOnly

Acceptance criteria

- The priority field is a dropdown with names, in filter bar and form.

Solution

Dropdown for priority (explicit annotation on the association, referring to the foreign key):

app/tickets/annotations/valuehelp.cds

```
annotate service.Tickets with {
  priority @(
    Common.ValueListWithFixedValues : true,
    Common.ValueList : {
      $Type          : 'Common.ValueListType',
      CollectionPath : 'Priorities',
      Parameters     : [
        { $Type : 'Common.ValueListParameterInOut',       LocalDataProperty : priority_code, ValueListProperty :
     'code' },
        { $Type : 'Common.ValueListParameterDisplayOnly', ValueListProperty : 'name' }
      ]
    }
  );
};
```

Guideline: dropdown for roughly up to 15 entries that never change often; a dialog for anything larger or searchable.

Story 4.3 - Standard value help with columns and search

Goal: Customer selection dialog with several columns.

Your tasks

1. Create a ValueList for customer_ID: collection Customers, search supported, return the ID, show name, e-mail and country as columns.
2. Show the customer as text first followed by the ID in brackets.
3. Make the customer name searchable in the dialog and verify it with a partial name.
4. Check the sort order in the dialog and set a default sort if possible.

Where to look / topics to explore

- Common.ValueList: CollectionPath, SearchSupported, Parameters
- Common.TextArrangement #TextFirst

Acceptance criteria

- The dialog lists the four customers with three columns.
- Selecting a row fills the field and shows the name.

Solution

Customer value help with columns and search:

app/tickets/annotations/valuehelp.cds

```
annotate service.Tickets with {
  customer @(
    Common.Text            : customer.name,
    Common.TextArrangement : #TextFirst,
    Common.ValueList : {
      $Type          : 'Common.ValueListType',
      CollectionPath : 'Customers',
      SearchSupported : true,
      Parameters     : [
```

```
        { $Type : 'Common.ValueListParameterInOut',       LocalDataProperty : customer_ID, ValueListProperty : '
    ID' },
        { $Type : 'Common.ValueListParameterDisplayOnly', ValueListProperty : 'name' },
        { $Type : 'Common.ValueListParameterDisplayOnly', ValueListProperty : 'email' },
        { $Type : 'Common.ValueListParameterDisplayOnly', ValueListProperty : 'country' }
      ]
    }
  );
};

annotate service.Customers with @cds.search : { name, email };
annotate service.Customers with { name @Common.Label : 'Name'; email @Common.Label : 'E-Mail'; country @Common.L
    abel : 'Country'; };
```

This replaces the TextOnly arrangement from Milestone 2; keep one definition per field (remove the old one to avoid confusion).

Story 4.4 - Out and Constant parameters

Goal: Value help that fills other fields and restricts rows.

Your tasks

1. On assignee_ID create a value help with agents. Add an **Out** parameter that copies the agent team into assigneeTeam when an agent is chosen.
2. Add a **Constant** parameter so only agents of one team are offered (decide the team and a business reason).
3. Test: choose an agent and watch assigneeTeam change in edit mode.
4. Write the difference between InOut, Out, In, Constant and DisplayOnly in your own words.

Where to look / topics to explore

- ValueListParameterOut
- ValueListParameterConstant

Acceptance criteria

- assigneeTeam is filled automatically.
- Only agents of the allowed team can be chosen.

Solution

Assignee with Out and Constant parameters:

app/tickets/annotations/valuehelp.cds

```
annotate service.Tickets with {
  assignee @(
    Common.Text            : assignee.name,
    Common.TextArrangement : #TextOnly,
    Common.ValueList : {
      $Type          : 'Common.ValueListType',
      CollectionPath : 'Agents',
      SearchSupported : true,
      Parameters     : [
        { $Type : 'Common.ValueListParameterInOut',       LocalDataProperty : assignee_ID,  ValueListProperty :
    'ID' },
        { $Type : 'Common.ValueListParameterDisplayOnly', ValueListProperty : 'name' },
        { $Type : 'Common.ValueListParameterOut',         LocalDataProperty : assigneeTeam, ValueListProperty :
    'team' },
        { $Type : 'Common.ValueListParameterConstant',    ValueListProperty : 'team', Constant : 'Support' }
      ]
    }
  );
};
```

Summary: **InOut** = field value goes in and the selected key comes back; **In** = filter only; **Out** = result copied into a field; **Constant** = fixed filter; **DisplayOnly** = extra column.

Story 4.5 - Dependent value help: category -> subcategory

Goal: Subcategories depend on the selected category.

Your tasks

1. Create a value help for category_code (dropdown) and for subcategory_code (dialog).
2. Make the subcategory value help filter by the current category using an **In** parameter.
3. Test in the filter bar and in the edit form. Describe what happens when the category is empty.
4. Handle the stale value: when the category changes, the old subcategory becomes invalid. Note the problem; you solve it with a side effect in Milestone 6.

Where to look / topics to explore

- ValueListParameterIn
- Value help refresh in the Side Effects documentation

Acceptance criteria

- Only subcategories of the chosen category appear.

Solution

Category dropdown and dependent subcategory:

app/tickets/annotations/valuehelp.cds

```
annotate service.Tickets with {
  category @(
    Common.Text : category.name, Common.TextArrangement : #TextOnly,
    Common.ValueListWithFixedValues : true,
    Common.ValueList : { $Type : 'Common.ValueListType', CollectionPath : 'Categories',
      Parameters : [
        { $Type : 'Common.ValueListParameterInOut',       LocalDataProperty : category_code, ValueListProperty :
     'code' },
        { $Type : 'Common.ValueListParameterDisplayOnly', ValueListProperty : 'name' } ] }
  );
  subcategory @(
    Common.Text : subcategory.name, Common.TextArrangement : #TextOnly,
    Common.ValueList : { $Type : 'Common.ValueListType', CollectionPath : 'Subcategories',
      Parameters : [
        { $Type : 'Common.ValueListParameterIn',          LocalDataProperty : category_code,    ValueListPropert
    y : 'category_code' },
        { $Type : 'Common.ValueListParameterInOut',       LocalDataProperty : subcategory_code, ValueListPropert
    y : 'code' },
        { $Type : 'Common.ValueListParameterDisplayOnly', ValueListProperty : 'name' } ] }
  );
};
```

With an empty category the In parameter has no value, so all subcategories are offered (a business decision: you may prefer to make subcategory read-only until a category is chosen with field control).

Story 4.6 - Value help in the filter bar and validation

Goal: Consistent value help everywhere.

Your tasks

1. Check every selection field has a sensible control: status, priority, category, assignee, customer.
2. Type a value that does not exist into the customer field. Observe the validation message.
3. Research the annotation that tells the framework to validate typed values against the value list.

Where to look / topics to explore

- Common.ValueListForValidation

Acceptance criteria

- An invalid customer ID is rejected with a message.

Solution

Validate typed values against the list:

app/tickets/annotations/valuehelp.cds

```
annotate service.Tickets with {
```

```
    customer @Common.ValueListForValidation : '';
    assignee @Common.ValueListForValidation : '';
  };
```

The empty string means "the unqualified value list is used for validation". Check the current vocabulary description; if the framework in your version already validates, the annotation may only be needed for qualified lists.

Story 4.7 - Stretch: several value helps for one field

Goal: Offer alternatives (for example search customers by name or by country).

Your tasks

1. Research qualified value lists and the relevant-qualifiers annotation.
2. Create two qualified value helps for customer and let the user switch.

Where to look / topics to explore

- Common.ValueList #qualifier
- Common.ValueListRelevantQualifiers

Acceptance criteria

- Both value helps are usable.

Solution

Outline: define `Common.ValueList #byName` (columns name, email) and `Common.ValueList #byCountry` (columns country, name, sorted by country) on customer, then add `Common.ValueListRelevantQualifiers` with the qualifiers and (optionally) an expression that decides which qualifier is relevant. The framework shows a selector in the value help dialog. See the Fiori elements feature showcase for a working example (search for "ValueListRelevantQualifiers").

Milestone checkpoint

Question	Answer
When do you use ValueListWithFixedValues?	Small stable lists (about 10-20 entries) where a dropdown is faster than a dialog.
What is the effect of an In parameter?	The value of the local property is used as a filter in the value help request.
Which parameter copies a value into another field?	ValueListParameterOut (or InOut for the field itself).

Milestone 5 - Actions and Dynamic UI

Item	Detail
Mini project	Row actions, toolbar actions, actions with parameters, enablement rules and hidden buttons, all driven by the data.
Level / effort	Intermediate to Advanced / 8-10 hours
Annotations practised	UI.DataFieldForAction, bound and unbound actions, action parameters with value help, Core.OperationAvailable, Common.IsActionCritical, UI.DeleteHidden, UI.CreateHidden, Common.FieldControl, Capabilities restrictions.

Learning outcomes

- Add actions in different places.
- Use parameter dialogs with value help.
- Make buttons and fields react to data state.

Story 5.1 - Implement three bound actions in the backend

Goal: Business logic for the UI to call.

Your tasks

1. Declare bound actions on Tickets: escalate (no parameter), changePriority (parameter: priority code), resolve (parameter: resolution text up to 200 characters).
2. Implement them in a service handler: escalate sets isEscalated and priority High; changePriority validates the code and updates; resolve closes the ticket, sets completion to 100 and stores the resolution; do not allow resolve on a closed ticket.
3. Return meaningful HTTP errors for invalid input.
4. Test the three actions with a request file before touching the UI.

Where to look / topics to explore

- Bound actions in a projection (actions block)
- req.params and req.reject

Acceptance criteria

- The three actions work through HTTP and reject invalid input.

Solution

Declare the actions (parameters annotated in 5.3):

srv/service.cds

```
@odata.draft.enabled
entity Tickets as projection on desk.Tickets actions {
  @Common.IsActionCritical : true
  action escalate();
  action changePriority(priority : String(1));
  action resolve(resolution : String(200));
};
```

Handler class (this file grows in later milestones):

srv/service.js

```
const cds = require('@sap/cds')
const { SELECT, UPDATE } = cds.ql

module.exports = class DeskService extends cds.ApplicationService {
  init() {
    const { Tickets } = this.entities
    const { Tickets: T, Priorities } = cds.entities('desk')
    const keyOf = req => req.params[0]?.ID ?? req.params[0]   // key of the row the action was called on

    this.on('escalate', Tickets, async req => {
      const ID = keyOf(req)
```

```
      const t = await SELECT.one.from(T).columns('status_code').where({ ID })
      if (!t) return req.reject(404, 'Ticket not found')
      if (t.status_code === 'C') return req.reject(409, 'Closed tickets cannot be escalated')
      await UPDATE(T).set({ isEscalated: true, priority_code: 'H' }).where({ ID })
    })

    this.on('changePriority', Tickets, async req => {
      const ID = keyOf(req)
      const code = req.data.priority
      if (!code || !(await SELECT.one.from(Priorities).where({ code })))
        return req.reject(400, 'Unknown priority ' + code)
      await UPDATE(T).set({ priority_code: code }).where({ ID })
    })

    this.on('resolve', Tickets, async req => {
      const ID = keyOf(req)
      const t = await SELECT.one.from(T).columns('status_code').where({ ID })
      if (!t) return req.reject(404, 'Ticket not found')
      if (t.status_code === 'C') return req.reject(409, 'Ticket is already closed')
      if (!req.data.resolution?.trim()) return req.reject(400, 'A resolution text is required')
      await UPDATE(T).set({ status_code: 'C', completion: 100, resolution: req.data.resolution }).where({ ID })
    })

    return super.init()
  }
}
```

Test URL pattern for a bound action on a draft-enabled entity: POST /odata/v4/desk/Tickets(ID=<uuid>,IsActiveEntity=true)/DeskService.escalate with an empty JSON body.

Story 5.2 - Show actions in the table

Goal: Buttons in the toolbar and inline in rows.

Your tasks

1. Add the three actions as buttons in the table toolbar.
2. Make escalate an inline (row) button with an icon.
3. Give the resolve action a text label and an emphasised style.
4. Select zero, one and many rows and note when each button is enabled.

Where to look / topics to explore

- UI.DataFieldForAction: Action, Label, Inline, IconUrl, Criticality

Acceptance criteria

- Toolbar has three buttons; escalate also appears in each row.

Solution

Actions in the LineItem (append to the existing array):

app/tickets/annotations/actions.cds

```
annotate service.Tickets with @(
  UI.LineItem : [
    ...,
    { $Type : 'UI.DataFieldForAction', Action : 'DeskService.escalate',
      Label : 'Escalate', Inline : true, IconUrl : 'sap-icon://alert', Criticality : #Negative },
    { $Type : 'UI.DataFieldForAction', Action : 'DeskService.changePriority', Label : 'Change priority' },
    { $Type : 'UI.DataFieldForAction', Action : 'DeskService.resolve', Label : 'Resolve',
      ![@UI.Emphasized] : true }
  ]
);
```

Bound actions need a selected row; the framework enables them when at least one row is selected (and the enablement rule, if any, is true).

Story 5.3 - Parameter dialogs with value help

Goal: changePriority asks for the priority in a dialog with a dropdown; resolve asks for a text.

Your tasks

1. Annotate the action parameters with labels and a fixed-values value help for the priority parameter.
2. Make the resolution a multi-line text and mandatory.
3. Add a default value for the priority parameter (research the annotation).
4. Test cancel and confirm in the dialog.

Where to look / topics to explore

- @title on parameters
- Common.ValueList on action parameters
- UI.ParameterDefaultValue
- UI.MultiLineText

Acceptance criteria

- The dialog shows a dropdown with names and a mandatory multi-line field.

Solution

Parameter annotations written inline in the action declaration:

srv/service.cds

```
entity Tickets as projection on desk.Tickets actions {
  @Common.IsActionCritical : true
  action escalate();

  action changePriority(
    @(
      title : 'New priority',
      UI.ParameterDefaultValue : 'M',
      Common.ValueListWithFixedValues : true,
      Common.ValueList : {
        $Type : 'Common.ValueListType', CollectionPath : 'Priorities',
        Parameters : [
          { $Type : 'Common.ValueListParameterInOut', LocalDataProperty : priority, ValueListProperty : 'code' }
  ,
          { $Type : 'Common.ValueListParameterDisplayOnly', ValueListProperty : 'name' } ]
      }
    )
    priority : String(1)
  );

  action resolve(
    @( title : 'Resolution', UI.MultiLineText : true, Common.FieldControl : #Mandatory )
    resolution : String(200)
  );
};
```

Story 5.4 - Enablement and confirmation

Goal: Buttons are available only when meaningful.

Your tasks

1. Disable resolve and changePriority for closed tickets using the operation availability annotation and the binding parameter.
2. Ask for confirmation before escalate (critical action annotation).
3. Look at $metadata and find the name of the binding parameter (it is needed in paths).

Where to look / topics to explore

- Core.OperationAvailable with $edmJson
- Common.IsActionCritical
- Binding parameter name

Test scenarios (do them all)

- Select a closed ticket: resolve disabled.
- Select a closed and an open ticket together: observe and explain.

Acceptance criteria

- Buttons follow the status.
- Escalate asks for confirmation.

Solution

Enablement depends on the status of the bound row (binding parameter `in`; verify the name in $metadata):

app/tickets/annotations/actions.cds

```
annotate service.Tickets with actions {
  resolve @(
    Core.OperationAvailable : { $edmJson : { $Ne : [ { $Path : 'in/status_code' }, 'C' ] } }
  );
  changePriority @(
    Core.OperationAvailable : { $edmJson : { $Ne : [ { $Path : 'in/status_code' }, 'C' ] } }
  );
};
```

Mixed selection: the button is enabled if the rule allows it for the selected context; the framework evaluates per context, so behaviour with a mixed selection depends on the UI5 version. Test and note the outcome. The confirmation for escalate comes from `@Common.IsActionCritical`.

Story 5.5 - Global action

Goal: A toolbar action without selection.

Your tasks

1. Create an unbound action reassignOverdue(agent) that moves all overdue, not closed tickets to an agent and returns the count.
2. Show it in the toolbar so it is enabled without selection.
3. Add a value help for the agent parameter.

Where to look / topics to explore

- Unbound actions and EntityContainer path in Action

Acceptance criteria

- The action works without selection and returns the count in a message.

Solution

Declare, annotate, implement:

actions (three files)

```
// srv/service.cds (inside the service)
action reassignOverdue(
  @( title : 'Reassign to', Common.ValueList : { $Type : 'Common.ValueListType', CollectionPath : 'Agents',
       Parameters : [ { $Type : 'Common.ValueListParameterInOut', LocalDataProperty : agent, ValueListProperty :
     'ID' },
                    { $Type : 'Common.ValueListParameterDisplayOnly', ValueListProperty : 'name' } ] } )
  agent : UUID
) returns Integer;

// app/tickets/annotations/actions.cds  (add to LineItem)
{ $Type : 'UI.DataFieldForAction', Action : 'DeskService.EntityContainer/reassignOverdue', Label : 'Reassign ove
    rdue' }

// srv/service.js (inside init)
this.on('reassignOverdue', async req => {
  const { Tickets: T } = cds.entities('desk')
  const today = new Date().toISOString().slice(0, 10)
  const n = await UPDATE(T).set({ assignee_ID: req.data.agent })
                           .where({ dueDate: { '<': today }, status_code: { '!=': 'C' } })
  req.notify(n + ' tickets reassigned')
```

```
    return n
  })
```

Story 5.6 - Hide and restrict standard operations dynamically

Goal: Delete, edit and fields react to data.

Your tasks

1. Hide the Delete button for closed tickets.
2. Make closed tickets non-editable using update restrictions with a path.
3. Make the estimated hours and hourly rate read-only for closed tickets using field control.
4. Hide the Create button for a role of your choice in a later milestone (research how; do not implement now).

Where to look / topics to explore

- UI.DeleteHidden
- Capabilities.UpdateRestrictions.Updatable
- Common.FieldControl
- UI.CreateHidden

Acceptance criteria

- Closed tickets cannot be deleted or edited in the UI.

Solution

Dynamic delete, update and field control:

app/tickets/annotations/actions.cds

```
annotate service.Tickets with @(
  UI.DeleteHidden : isClosed,
  Capabilities.UpdateRestrictions : { Updatable : isOpen }
);

annotate service.Tickets with {
  estimatedHours @Common.FieldControl : fieldControl;
  hourlyRate     @Common.FieldControl : fieldControl;
};
```

Hiding Create for a role: `UI.CreateHidden` accepts a Boolean path; role-based hiding needs a property computed from the user (set in an after READ handler) or the CAP `@restrict` annotation (the button is then removed automatically because the service forbids it).

Milestone checkpoint

Question	Answer
How do you find the binding parameter name?	In $metadata at the action definition (CAP default is "in").
Difference between DataFieldForAction with and without Inline?	Inline shows a button in each row; otherwise in the toolbar for selected rows.

Milestone 6 - Side Effects

Item	Detail
Mini project	The page keeps itself consistent: calculated fields, dependent fields and statuses refresh after changes and actions.
Level / effort	Advanced / 8-10 hours
Annotations practised	Common.SideEffects with SourceProperties, SourceEntities, TargetProperties, TargetEntities, action side effects, qualifiers, debugging in the network tab.

Learning outcomes

- Know what Fiori elements refreshes by default and what not.
- Write side effects for fields, entities and actions.
- Debug side effects with browser tools.

Story 6.1 - Baseline: see the problem first

Goal: Understand why side effects are needed.

Your tasks

1. Edit a ticket in the Object Page. Change estimated hours and hourly rate. Observe that the cost field does not change until you save.
2. Change the category: the subcategory keeps the old value.
3. Read the Side Effects chapter of the Fiori elements documentation. Write which refreshes happen automatically and which need annotation.

Where to look / topics to explore

- Fiori elements docs: Side Effects

Acceptance criteria

- You documented three situations where the UI is inconsistent without side effects.

Solution

Typical inconsistencies: (1) cost not recalculated while editing (calculated on the server); (2) subcategory not cleared after category change; (3) status criticality and read-only state not updated after status change; (4) contact card of the old customer. The framework only refreshes by default: list after create/delete/draft operations, and the changed field itself.

Story 6.2 - Recalculate cost when inputs change

Goal: cost updates immediately when hours or rate change.

Your tasks

1. Add a side effect with source properties estimatedHours and hourlyRate and target property cost.
2. Test in the Object Page: change hours, leave the field, watch the network tab for the request.
3. Explain why no JavaScript was needed (cost is a calculated element).

Where to look / topics to explore

- Common.SideEffects with qualifier

Acceptance criteria

- Cost changes after leaving the field.
- You can show the request in the network tab.

Solution

Cost recalculation:

```
app/tickets/annotations/sideeffects.cds
annotate service.Tickets with @(
  Common.SideEffects #CostRecalc : {
    SourceProperties : [ estimatedHours, hourlyRate ],
    TargetProperties : [ 'cost' ]
  }
);
```

No JavaScript: `cost` is a calculated element, so the server returns the new value in the side-effect read.

Story 6.3 - Reset dependent field

Goal: Changing the category clears the subcategory and refreshes its value help.

Your tasks

1. In the service add a handler that clears subcategory when category changes in a draft.
2. Add a side effect with source category and target subcategory (key and text).
3. Add a validation on save: the subcategory must belong to the category.
4. Test sequence: choose category, choose subcategory, change category.

Where to look / topics to explore

- PATCH handler on draft entity
- before SAVE

Acceptance criteria

- Subcategory is empty after changing the category.
- Saving a mismatching pair is rejected.

Solution

Backend (draft PATCH and SAVE validation):

```
srv/service.js
this.before('PATCH', Tickets.drafts, req => {
  if ('category_code' in req.data) req.data.subcategory_code = null
  if (req.data.status_code === 'C') req.data.completion = 100
})

this.before('SAVE', Tickets, async req => {
  const { Subcategories } = cds.entities('desk')
  const { category_code, subcategory_code } = req.data
  if (subcategory_code) {
    const sc = await SELECT.one.from(Subcategories).where({ code: subcategory_code })
    if (!sc || sc.category_code !== category_code)
      req.error({ status: 400, target: 'subcategory_code', message: 'Subcategory does not belong to the category
    ' })
  }
})
```

Annotation:

```
app/tickets/annotations/sideeffects.cds
annotate service.Tickets with @(
  Common.SideEffects #CategoryChange : {
    SourceProperties : [ category_code ],
    TargetProperties : [ 'subcategory_code', 'subcategory/name' ]
  }
);
```

Story 6.4 - Status change updates several fields

Goal: One change, many targets.

Your tasks

1. When the status becomes Closed the backend sets completion to 100.

2. Add a side effect with source status and targets: status criticality, field control, open and closed flags, completion.
3. Verify that fields become read-only or editable immediately.
4. Try the wildcard target and explain why a precise list is better.

Where to look / topics to explore

- TargetProperties with *

Acceptance criteria

- Closing the ticket in the form shows completion 100 and read-only fields without saving.

Solution

One source, several targets:

app/tickets/annotations/sideeffects.cds

```
annotate service.Tickets with @(
  Common.SideEffects #StatusChange : {
    SourceProperties : [ status_code ],
    TargetProperties : [ 'statusCriticality', 'isOpen', 'isClosed', 'fieldControl', 'completion' ]
  }
);
```

The backend part (completion = 100) is the PATCH handler from story 6.3. A wildcard target (*) refreshes everything of the entity: simple, but heavy and may overwrite what the user is typing; prefer precise lists.

Story 6.5 - Refresh related entities

Goal: Contact and worklogs refresh when their parent selection changes.

Your tasks

1. When the customer changes, refresh the customer navigation entity so the contact card changes.
2. Add a side effect that refreshes the worklogs table when the assignee changes (invent a rule and implement a small handler).
3. Note the difference between target properties and target entities.

Where to look / topics to explore

- TargetEntities, SourceEntities

Acceptance criteria

- Customer contact data changes when the customer changes.

Solution

Entity targets:

app/tickets/annotations/sideeffects.cds

```
annotate service.Tickets with @(
  Common.SideEffects #CustomerChange : {
    SourceProperties : [ customer_ID ],
    TargetEntities   : [ customer ]
  },
  Common.SideEffects #AssigneeChange : {
    SourceProperties : [ assignee_ID ],
    TargetEntities   : [ worklogs ]
  }
);
```

Target properties reload single fields (cheap); target entities reload whole navigation targets (tables, contact cards). For the worklog rule, a sensible backend rule is: on assignee change append a worklog line "Assigned to <name>" in a before SAVE handler.

Story 6.6 - Side effects on actions

Goal: After an action the visible row updates.

Your tasks

1. Add side effects to escalate, changePriority and resolve so the affected fields are reloaded in the list.
2. Find the way to refer to the bound entity inside the target paths.
3. Test in the list report: select, run action, watch the row change without a full reload.

Where to look / topics to explore

- Common.SideEffects on actions
- Binding parameter path prefix

Acceptance criteria

- The row updates immediately after each action.

Solution

Side effects on actions (binding parameter name `in`):

app/tickets/annotations/sideeffects.cds

```
annotate service.Tickets with actions {
  escalate @Common.SideEffects : {
    TargetProperties : [ 'in/isEscalated', 'in/priority_code', 'in/priorityCriticality' ] };
  changePriority @Common.SideEffects : {
    TargetProperties : [ 'in/priority_code', 'in/priorityCriticality' ] };
  resolve @Common.SideEffects : {
    TargetProperties : [ 'in/status_code', 'in/statusCriticality', 'in/completion',
                         'in/isOpen', 'in/isClosed', 'in/resolution' ] };
};
```

If you changed the binding parameter name with `cds.odata.bindingparameter.name` (for example to `_it`), use that prefix instead of `in/`.

Story 6.7 - Debugging and performance

Goal: Be able to find out why a side effect does not work.

Your tasks

1. Write a checklist: annotation compiled? property names correct? draft or active? network request visible? backend returns the field?
2. Create a deliberately wrong side effect (wrong property) and find the error.
3. Review: remove side effects that refresh too much.

Where to look / topics to explore

- Browser developer tools, network tab, OData $select

Acceptance criteria

- Your checklist exists and you fixed the broken example.

Solution

Checklist:

1) Does the annotation compile and appear in $metadata under the entity? 2) Are source and target names the real property names (foreign keys end with _code or _ID)? 3) In edit mode the entity is the draft: does the request go to the draft URL (IsActiveEntity=false)? 4) Does the network tab show a GET with the expected $select after leaving the field? 5) Does the response contain the field with a new value? 6) Is another side effect or a handler overwriting it? 7) Is the target a navigation property (then use TargetEntities)?

Milestone checkpoint

Question	Answer
Source versus target?	Source = what the user changes; target = what must be refreshed from the server.
Why prefer property targets to entity targets?	Smaller request and less flicker.
Which side effects does the framework do on its own?	Create, delete, draft create/discard/activate refresh the list binding.

Milestone 7 - Multiple Views, Variants and Presentation

Item	Detail
Mini project	The list report gets tabs (All, Open, Escalated, Closed) with counts, defined sort and grouping, variant management and a table/chart switch.
Level / effort	Advanced / 8-12 hours
Annotations practised	UI.SelectionVariant, UI.PresentationVariant, UI.SelectionPresentationVariant, manifest views and showCounts, variantManagement, UI.Chart, Aggregation annotations.

Learning outcomes

- Build several views on one list report.
- Define default sort and grouping.
- Understand variants (user level) versus views (developer level).

Story 7.1 - Variant management

Goal: Understand the three levels and set the right one.

Your tasks

1. Read the documentation on variant management: None, Page, Control.
2. Test each value in the manifest and note what the user can save and where the variant selector appears.
3. Create two personal variants in the running app (for example "My escalations" and "High priority open") and set one as default.
4. Decide the value you keep and justify (hint: it interacts with multiple views).

Where to look / topics to explore

- sap.fe.templates.ListReport settings: variantManagement

Acceptance criteria

- You documented the three values and chose one with a reason.

Solution

None = no variant selector; Page = one variant selector for the whole page (filter bar and table share the variant); Control = separate variant management for the filter bar and for each table (needed when several views are used). Setting in the manifest:

```
manifest.json (settings of the list report target)
 "sap.fe.templates.ListReport": { "settings": { "variantManagement": "Control" } }
```

Story 7.2 - Selection variants

Goal: Predefined filter sets.

Your tasks

1. Define selection variants: All (no restriction), Open (status New, In Progress or Waiting), Escalated (isEscalated true), Closed (status Closed).
2. Learn the meaning of Sign, Option, Low and High in select options.
3. Define a High priority variant using a range on priority.
4. Check how the variants show in the generated metadata.

Where to look / topics to explore

- UI.SelectionVariant, UI.SelectOptionType, UI.SelectionRangeType

Acceptance criteria

- Four selection variants compile and appear in $metadata.

Solution

Selection variants (Sign I = include, Option EQ = equals, Low/High = bounds; BT = between):

app/tickets/annotations/variants.cds

```
annotate service.Tickets with @(
  UI.SelectionVariant #All : { Text : 'All' },
  UI.SelectionVariant #Open : {
    Text : 'Open',
    SelectOptions : [{ PropertyName : status_code, Ranges : [
      { Sign : #I, Option : #EQ, Low : 'N' },
      { Sign : #I, Option : #EQ, Low : 'I' },
      { Sign : #I, Option : #EQ, Low : 'W' } ] }]
  },
  UI.SelectionVariant #Escalated : {
    Text : 'Escalated',
    SelectOptions : [{ PropertyName : isEscalated, Ranges : [{ Sign : #I, Option : #EQ, Low : true }] }]
  },
  UI.SelectionVariant #Closed : {
    Text : 'Closed',
    SelectOptions : [{ PropertyName : status_code, Ranges : [{ Sign : #I, Option : #EQ, Low : 'C' }] }]
  },
  UI.SelectionVariant #HighPrio : {
    Text : 'High priority',
    SelectOptions : [{ PropertyName : priority_code, Ranges : [{ Sign : #I, Option : #EQ, Low : 'H' }] }]
  }
);
```

Story 7.3 - Presentation variants

Goal: Sorting, grouping and visible columns per view.

Your tasks

1. Define a presentation variant that sorts by priority criticality then due date.
2. Define one that groups by category and sorts by due date.
3. Define a presentation variant with a limited set of visualizations (research MaxItems and RequestAtLeast).
4. Test grouping in the UI and compare with the table settings group option.

Where to look / topics to explore

- UI.PresentationVariant: SortOrder, GroupBy, Visualizations, RequestAtLeast

Acceptance criteria

- Sorting and grouping happen on first load without user action.

Solution

Presentation variants:

app/tickets/annotations/variants.cds

```
annotate service.Tickets with @(
  UI.PresentationVariant #ByUrgency : {
    SortOrder : [ { Property : priorityCriticality, Descending : false },
                  { Property : dueDate, Descending : false } ],
    Visualizations : [ '@UI.LineItem' ]
  },
  UI.PresentationVariant #ByCategory : {
    GroupBy : [ category_code ],
    SortOrder : [ { Property : dueDate, Descending : false } ],
    Visualizations : [ '@UI.LineItem' ]
  }
);
```

Table settings allow users to group at runtime; a presentation variant defines the initial grouping and sorting.

Story 7.4 - Tabs: selection-presentation variants and manifest

Goal: Four tabs with counts.

Your tasks

1. Combine selection and presentation variants into UI.SelectionPresentationVariant annotations: All, Open, Escalated, Closed.
2. Register them as views in the list report settings of the manifest and enable counts.
3. Test counts when filters in the filter bar change.
4. Test behaviour with the default status filter from Milestone 3 and resolve conflicts (decide what to remove).
5. If you get a variant-management error, read the documentation and fix the manifest accordingly.

Where to look / topics to explore

- sap.fe.templates.ListReport options.settings.views.paths and showCounts
- Blog: configure multiple views on list report tables

Test scenarios (do them all)

- Click each tab and watch the filters and counts.
- Change a filter field and see how counts react.

Acceptance criteria

- Four tabs show correct counts.

Solution

Selection-presentation variants (one per tab; the selection part can be inline or reference a qualifier):

app/tickets/annotations/variants.cds

```
annotate service.Tickets with @(
  UI.SelectionPresentationVariant #TabAll : {
    $Type : 'UI.SelectionPresentationVariantType', Text : 'All',
    SelectionVariant    : { $Type : 'UI.SelectionVariantType', SelectOptions : [] },
    PresentationVariant : { $Type : 'UI.PresentationVariantType',
      SortOrder : [{ Property : dueDate, Descending : false }], Visualizations : ['@UI.LineItem'] }
  },
  UI.SelectionPresentationVariant #TabOpen : {
    $Type : 'UI.SelectionPresentationVariantType', Text : 'Open',
    SelectionVariant : { $Type : 'UI.SelectionVariantType', Text : 'Open',
      SelectOptions : [{ PropertyName : status_code, Ranges : [
        { Sign : #I, Option : #EQ, Low : 'N' }, { Sign : #I, Option : #EQ, Low : 'I' }, { Sign : #I, Option : #E
    Q, Low : 'W' } ] }] },
    PresentationVariant : { $Type : 'UI.PresentationVariantType',
      SortOrder : [{ Property : priorityCriticality, Descending : false }], Visualizations : ['@UI.LineItem'] }
  },
  UI.SelectionPresentationVariant #TabEscalated : {
    $Type : 'UI.SelectionPresentationVariantType', Text : 'Escalated',
    SelectionVariant : { $Type : 'UI.SelectionVariantType', Text : 'Escalated',
      SelectOptions : [{ PropertyName : isEscalated, Ranges : [{ Sign : #I, Option : #EQ, Low : true }] }] },
    PresentationVariant : { $Type : 'UI.PresentationVariantType', Visualizations : ['@UI.LineItem'] }
  },
  UI.SelectionPresentationVariant #TabClosed : {
    $Type : 'UI.SelectionPresentationVariantType', Text : 'Closed',
    SelectionVariant : { $Type : 'UI.SelectionVariantType', Text : 'Closed',
      SelectOptions : [{ PropertyName : status_code, Ranges : [{ Sign : #I, Option : #EQ, Low : 'C' }] }] },
    PresentationVariant : { $Type : 'UI.PresentationVariantType', Visualizations : ['@UI.LineItem'] }
  }
);
```

Manifest: register the tabs in the list report target:

manifest.json (sap.ui5.routing.targets)

```
"TicketsList": { "type": "Component", "id": "TicketsList", "name": "sap.fe.templates.ListReport",
  "options": { "settings": {
    "contextPath": "/Tickets",
    "variantManagement": "Control",
    "views": {
      "showCounts": true,
      "paths": [
        { "key": "all",       "annotationPath": "com.sap.vocabularies.UI.v1.SelectionPresentationVariant#TabAll"
```

```
      },
         { "key": "open",      "annotationPath": "com.sap.vocabularies.UI.v1.SelectionPresentationVariant#TabOpen
      " },
         { "key": "escalated", "annotationPath": "com.sap.vocabularies.UI.v1.SelectionPresentationVariant#TabEsca
      lated" },
         { "key": "closed",    "annotationPath": "com.sap.vocabularies.UI.v1.SelectionPresentationVariant#TabClos
      ed" }
        ] } } } }
```

Conflict with the default status filter of Milestone 3: remove `FilterDefaultValue` for status when you use tabs, otherwise the Open tab and the default filter intersect and counts confuse users.

Story 7.5 - Table and chart in one list report

Goal: Switch between table and chart view.

Your tasks

1. Define a column chart: category on the axis, cost as measure.
2. Declare the aggregation support needed for the chart (research which annotations and whether draft-enabled entities need special care).
3. Add a selection-presentation variant that uses the chart as visualization and register table and chart as primary and secondary views.
4. Document the limitation you meet (this story is hard; keep notes even if you do not finish).

Where to look / topics to explore

- UI.Chart
- Aggregation.CustomAggregate, Aggregation.ApplySupported
- Feature showcase: multiple table mode

Acceptance criteria

- A chart renders or you documented precisely why not and what you tried.

Solution

Chart, aggregation and the two-view configuration (this works best on a non-draft entity; if your draft entity does not support the grouped read, document it and use a separate analytical view as an exercise):

app/tickets/annotations/variants.cds

```
annotate service.Tickets with @(
  Aggregation.CustomAggregate #cost : 'Edm.Decimal',
  UI.Chart #CostByCategory : {
    $Type : 'UI.ChartDefinitionType', Title : 'Cost by category', ChartType : #Column,
    Dimensions : [ category_code ],
    DimensionAttributes : [{ $Type : 'UI.ChartDimensionAttributeType', Dimension : category_code, Role : #Catego
    ry }],
    Measures : [ cost ],
    MeasureAttributes : [{ $Type : 'UI.ChartMeasureAttributeType', Measure : cost, Role : #Axis1 }]
  },
  UI.SelectionPresentationVariant #ChartView : {
    $Type : 'UI.SelectionPresentationVariantType', Text : 'Chart',
    SelectionVariant : { $Type : 'UI.SelectionVariantType', SelectOptions : [] },
    PresentationVariant : { $Type : 'UI.PresentationVariantType', Visualizations : ['@UI.Chart#CostByCategory']
    }
  }
);
```

Manifest: table as primary and chart as secondary view:

manifest.json

```
"views": { "paths": [{
   "primary":   [{ "annotationPath": "com.sap.vocabularies.UI.v1.SelectionPresentationVariant#TabAll" }],
   "secondary": [{ "annotationPath": "com.sap.vocabularies.UI.v1.SelectionPresentationVariant#ChartView" }],
   "defaultPath": "primary" }] }
```

Typical blockers: the service must support `$apply` (groupby/aggregate); calculated elements and draft handling can limit it. See the feature showcase (multiple table mode) for the supported shape in your UI5 version.

Story 7.6 - Stretch: totals and grouping in the table

Goal: Totals per currency.

Your tasks

1. Research table types that show totals and what annotation or setting enables them.
2. Try to show the total of cost in the table footer.

Where to look / topics to explore

- AnalyticalTable
- Aggregation annotations

Acceptance criteria

- You know the preconditions for totals.

Solution

Totals need an analytical-capable table (AnalyticalTable) and aggregation support on the service (`$apply`), plus an annotation that marks the measure as aggregatable (Aggregation.CustomAggregate / Analytics.AggregatedProperty) and a currency/unit property to group by. In Fiori elements V4 the table setting `type: "AnalyticalTable"` together with these annotations shows grand totals in the footer.

Milestone checkpoint

Question	Answer
View versus variant?	A view (tab) is defined by the developer in annotations/manifest; a variant is saved by the user.
What does showCounts do?	It requests a count per tab with that tab's filter.

Milestone 8 - Table and Page Configuration (manifest.json)

Item	Detail
Mini project	Everything you can tune without annotations: table type, selection, creation, export, paste, personalisation, and a custom column.
Level / effort	Advanced / 6-8 hours
Annotations practised	tableSettings, creationMode, selectionMode, enableExport, enablePaste, personalization, condensed layout, custom columns, initialLoad, filter bar settings.

Learning outcomes

- Know the manifest settings that complement annotations.
- Add a custom column with a fragment.
- Decide table type by use case.

Story 8.1 - Table types

Goal: Choose the right table.

Your tasks

1. Try ResponsiveTable, GridTable and AnalyticalTable (and TreeTable if you wish) for the list.
2. For each write: look and feel, performance, features (grouping, totals, personalisation) and restrictions.
3. Choose one for HR Ticket use and justify.

Where to look / topics to explore

- LineItem controlConfiguration: tableSettings.type

Acceptance criteria

- A comparison table exists with a final choice.

Solution

Comparison guide: **ResponsiveTable** (default; best on phones, row-based popin, fewest restrictions); **GridTable** (many rows, virtual scrolling, fixed column layout, no popin); **AnalyticalTable** (grouping, totals, needs aggregation support on the service); **TreeTable** (hierarchies, needs a hierarchy annotation). For service-desk tickets: ResponsiveTable (mobile agents) or GridTable (dispatch screen).

```
manifest.json (options.settings of the list report)
 "controlConfiguration": { "@com.sap.vocabularies.UI.v1.LineItem": { "tableSettings": { "type": "GridTable" } } }
```

Story 8.2 - Selection, creation and export

Goal: Behaviour settings.

Your tasks

1. Set selection mode to multi and enable select all.
2. Set creation mode: create in a new page, then inline, and compare (inline needs more conditions: research them).
3. Enable or disable export to spreadsheet and test the exported file. Is the export honouring filters?
4. Enable paste and test pasting rows from a spreadsheet. Write how this differs from Milestone 9 (Excel upload).

Where to look / topics to explore

- tableSettings: selectionMode, creationMode, enableExport, enablePaste, selectAll

Acceptance criteria

- Each option was switched and the result documented.

Solution

manifest.json (options.settings of the list report)

```
"controlConfiguration": {
  "@com.sap.vocabularies.UI.v1.LineItem": {
    "tableSettings": {
      "type": "ResponsiveTable",
      "selectionMode": "Multi",
      "selectAll": true,
      "creationMode": { "name": "NewPage" },
      "enableExport": true,
      "enablePaste": false
    } } }
```

Inline creation needs `"name": "Inline"` or `"InlineCreationRows"` and an entity that supports it (not draft-enabled in some versions). Export honours the current filters and the visible columns. Paste adds rows from the clipboard in the table; the Excel upload of Milestone 9 reads a file, validates server-side and reports per row.

Story 8.3 - Personalisation and layout

Goal: User-driven table features.

Your tasks

1. Control which personalisation features (column, sort, filter, group) are available.
2. Switch condensed layout on and compare density on desktop.
3. Make column widths include the header text.
4. Add a full screen option.

Where to look / topics to explore

- tableSettings: personalization, condensedTableLayout, widthIncludingColumnHeader, enableFullScreen

Acceptance criteria

- Personalisation is limited to what you chose.

Solution

manifest.json (tableSettings)

```
"tableSettings": {
  "personalization": { "column": true, "sort": true, "filter": false, "group": false },
  "condensedTableLayout": true,
  "widthIncludingColumnHeader": true,
  "enableFullScreen": true
}
```

Story 8.4 - Page-level settings

Goal: Filter bar and loading behaviour.

Your tasks

1. Control initial load (Enabled, Disabled, Auto) and test with a required filter.
2. Hide the filter bar entirely and use a hidden filter bar; document when this makes sense.
3. Set live mode or Go button behaviour for the filter bar if available in your version.

Where to look / topics to explore

- sap.fe.templates.ListReport settings: initialLoad, hideFilterBar, useHiddenFilterBar
- filterBar settings

Acceptance criteria

- You can list five page-level settings and what they do.

Solution

manifest.json (page settings)

```
"sap.fe.templates.ListReport": { "settings": {
   "initialLoad": "Disabled",
   "variantManagement": "Control" } }
```

Five useful page settings: `initialLoad`, `variantManagement`, `hideFilterBar`, `useHiddenFilterBar` (filter bar hidden but filters still applied, e.g. for table-only pages), `defaultTemplateAnnotationPath`. With a required filter, `initialLoad: Disabled` avoids reading all data before the user picks a status.

Story 8.5 - Custom column with a fragment

Goal: An SLA column that shows Overdue or On time.

Your tasks

1. Create an XML fragment that displays a status text and colour based on the due date and ticket status.
2. Register it as a custom column in the manifest, placed after the due date column, with a header text and horizontal alignment.
3. Make it available by default and test personalisation.
4. Think about what is evaluated client-side and what would be better as a backend field.

Where to look / topics to explore

- controlConfiguration LineItem columns: header, position (placement, anchor), template, availability
- Expression binding in XML

Acceptance criteria

- The column shows red Overdue for late open tickets and nothing for closed tickets.

Solution

Fragment (client-side evaluation with an expression binding):

webapp/ext/fragment/SlaColumn.fragment.xml

```
<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">
  <ObjectStatus
    text="{= ${isOpen} &amp;&amp; ${dueDate} &amp;&amp; ${dueDate} &lt; new Date().toISOString().slice(0,10) ? '
    Overdue' : ''}"
    state="{= ${isOpen} &amp;&amp; ${dueDate} &amp;&amp; ${dueDate} &lt; new Date().toISOString().slice(0,10) ?
    'Error' : 'None'}"
    inverted="true" />
</core:FragmentDefinition>
```

Manifest registration:

manifest.json

```
"controlConfiguration": { "@com.sap.vocabularies.UI.v1.LineItem": { "columns": {
  "SlaColumn": {
    "header": "SLA",
    "position": { "placement": "After", "anchor": "DataField::dueDate" },
    "template": "desk.tickets.ext.fragment.SlaColumn",
    "availability": "Default",
    "horizontalAlign": "Center",
    "properties": [ "dueDate", "isOpen" ]
  } } } }
```

Better design: a calculated backend field `isOverdue` that can be sorted, filtered and exported; the fragment then only shows it. The `properties` array makes the framework request the needed fields.

Milestone checkpoint

Question	Answer
Which settings live in the manifest and not in annotations?	Table type, selection and creation mode, export/paste, personalisation, load behaviour, custom extensions.
Why prefer a backend field over a client-side custom column?	Sorting, filtering and export work only with real properties.

Milestone 9 - Excel Upload that Creates Table Data

Item	Detail
Mini project	A toolbar button on the List Report lets users upload an .xlsx file; the rows are validated and become tickets; errors are reported per row.
Level / effort	Advanced / 10-14 hours
Annotations practised	Custom action in manifest, JavaScript handler, dialog with file uploader, calling a CAP action, parsing Excel on the server, validation, partial success reporting, refreshing the table, alternative: third-party spreadsheet importer component.

Learning outcomes

- Compare a ready-made component with your own implementation.
- Implement end-to-end Excel upload in a Fiori elements app.
- Design validation and error reporting for bulk data.

Story 9.1 - Compare the two approaches

Goal: Choose deliberately.

Your tasks

1. Approach A: the open-source UI5 spreadsheet importer component. Read its documentation and demo. Note supported scenarios and the **licence terms of recent major versions** (non-production versus production).
2. Approach B: your own custom action, dialog, and a CAP action that parses the file.
3. Compare: effort, flexibility, licence, error reporting, template download, draft handling, maintenance.
4. Decide: you implement B and also try A in a copy of the project.

Where to look / topics to explore

- docs.spreadsheet-importer.com
- SAP Community blog about mass upload from a spreadsheet into a Fiori app

Acceptance criteria

- A written comparison and the decision exist.

Solution

Suggested comparison: **A (importer component)** is fast to integrate, supports many scenarios, has a template download and dialog out of the box; but check the licence of the version you use (recent major versions are free for non-production use and need a licence for production), and you depend on a third party. **B (own implementation)** costs more effort but gives full control over validation, lookups by e-mail, error messages and security, and has no licence dependency; you own the maintenance.

Story 9.2 - Define the template and the rules

Goal: A contract between user and system.

Your tasks

1. Create an Excel template Ticket_Upload_Template.xlsx with columns: title, description, category, subcategory, priority, channel, customerEmail, assigneeEmail, dueDate, estimatedHours, hourlyRate, currency.
2. Write the rules per column: mandatory or optional, allowed values (codes), format (date), range (hours 0-999), length limits.
3. Decide how a customer and an agent are identified (by e-mail) and what happens if they are not found.
4. Decide: all rows or nothing, or valid rows only with an error report. Write the decision and why.
5. Prepare three test files: valid (10 rows), mixed (valid and invalid rows), broken (wrong headers).

Where to look / topics to explore

- Think like a user: what error message would help you fix the file?

Acceptance criteria

- Template, rules table and three test files exist.

Solution

Rules table (example):

```
rules
Column          Mandatory  Rule
title           yes        1-120 characters
description     no         up to 1000 characters
category        yes        existing category code (HW, SW, NET, ACC)
subcategory     no         existing code AND belongs to the category
priority        yes        H, M or L
channel         no         EMAIL, PHONE, WEB or CHAT
customerEmail   yes        must match an existing customer
assigneeEmail   no         must match an existing agent
dueDate         no         valid date, yyyy-mm-dd, not before 2000-01-01
estimatedHours  no         number 0 - 999
hourlyRate      no         number >= 0
currency        no         EUR, USD or INR
```

Decision recommended for learning: insert the valid rows and report the invalid ones (partial success), because it shows more of the UX work. For financial or master data prefer all-or-nothing.

Story 9.3 - Backend action to receive and process the file

Goal: A CAP action that accepts the file and returns a result.

Your tasks

1. Declare an unbound action uploadExcel(fileName, content) returning inserted count, failed count and a list of errors with row number and message. Choose a transport for the file content (research base64 in JSON versus multipart) and justify.
2. Install a maintained Excel library for Node.js (research options and their security status; avoid unmaintained versions).
3. Implement: validate file name and size, read the first worksheet, verify headers, read rows, validate each row against your rules, resolve customer and agent by e-mail, generate ticket numbers, insert valid rows.
4. Return a summary; never fail the whole request because of one bad row (according to your decision).
5. Test with your three files through an HTTP client before building any UI.

Where to look / topics to explore

- LargeString type in CDS
- exceljs or another maintained library
- INSERT.into(...).entries

Test scenarios (do them all)

- Valid file: 10 inserted.
- Mixed file: correct counts and row numbers.
- Broken file: clear message about missing columns.
- Empty sheet, wrong extension, 2,000 rows.

Acceptance criteria

- The action returns correct summaries for all test files.

Solution

Declaration:

srv/service.cds

```
// inside service DeskService
type UploadError  { row : Integer; message : String; }
type UploadResult { inserted : Integer; failed : Integer; errors : array of UploadError; }
action uploadExcel(fileName : String, content : LargeString) returns UploadResult;
```

Library: install exceljs (maintained, supports .xlsx); verify its status before you use it in production.

terminal

```
npm install exceljs
```

Implementation (inside init of the service class; add `const ExcelJS = require('exceljs')` at the top):

srv/service.js

```
this.on('uploadExcel', async req => {
  const { fileName, content } = req.data
  if (!/\.xlsx$/i.test(fileName || '')) return req.reject(400, 'Only .xlsx files are supported')
  if (!content) return req.reject(400, 'The file is empty')

  const wb = new ExcelJS.Workbook()
  await wb.xlsx.load(Buffer.from(content, 'base64'))
  const ws = wb.worksheets[0]
  if (!ws) return req.reject(400, 'The workbook has no worksheet')

  // 1. header row -> column index map
  const header = {}
  ws.getRow(1).eachCell((cell, col) => { header[String(cell.value).trim()] = col })
  const required = ['title', 'category', 'priority', 'customerEmail']
  const missing = required.filter(h => !(h in header))
  if (missing.length) return req.reject(400, 'Missing columns: ' + missing.join(', '))
  if (ws.rowCount > 1001) return req.reject(400, 'Maximum 1000 rows per file')

  // 2. lookups
  const { Tickets, Customers, Agents, Categories, Subcategories, Priorities, Channels } = cds.entities('desk')
  const byEmail = async E => Object.fromEntries((await SELECT.from(E).columns('ID', 'email', 'team')).map(r => [
    r.email?.toLowerCase(), r]))
  const customers = await byEmail(Customers)
  const agents = await byEmail(Agents)
  const codes = async E => new Set((await SELECT.from(E).columns('code')).map(r => r.code))
  const [cats, prios, chans] = [await codes(Categories), await codes(Priorities), await codes(Channels)]
  const subs = Object.fromEntries((await SELECT.from(Subcategories).columns('code', 'category_code')).map(r => [
    r.code, r.category_code]))

  const cellText = (row, name) => {
    const v = row.getCell(header[name] || 9999).value
    if (v == null) return ''
    if (v instanceof Date) return v.toISOString().slice(0, 10)
    if (typeof v === 'object') return String(v.text ?? v.result ?? '')   // rich text, hyperlink, formula
    return String(v).trim()
  }

  // 3. validate row by row
  const errors = [], valid = []
  ws.eachRow((row, n) => {
    if (n === 1) return
    const r = n, err = m => errors.push({ row: r, message: m })
    const title = cellText(row, 'title')
    if (!title) return                                      // ignore completely empty rows
    const d = {
      title, description: cellText(row, 'description'),
      category_code: cellText(row, 'category'), subcategory_code: cellText(row, 'subcategory') || null,
      priority_code: cellText(row, 'priority'), channel_code: cellText(row, 'channel') || null,
      dueDate: cellText(row, 'dueDate') || null, currency_code: cellText(row, 'currency') || null,
      status_code: 'N', isEscalated: false, completion: 0
    }
    if (title.length > 120) err('title longer than 120 characters')
    if (!cats.has(d.category_code)) err('unknown category ' + d.category_code)
    if (d.subcategory_code && subs[d.subcategory_code] !== d.category_code) err('subcategory does not belong to
    the category')
    if (!prios.has(d.priority_code)) err('unknown priority ' + d.priority_code)
    if (d.channel_code && !chans.has(d.channel_code)) err('unknown channel ' + d.channel_code)
    const cust = customers[cellText(row, 'customerEmail').toLowerCase()]
    if (!cust) err('customer not found: ' + cellText(row, 'customerEmail')); else d.customer_ID = cust.ID
    const ae = cellText(row, 'assigneeEmail').toLowerCase()
    if (ae) { const a = agents[ae]; if (!a) err('agent not found: ' + ae); else { d.assignee_ID = a.ID; d.assign
    eeTeam = a.team } }
    if (d.dueDate && Number.isNaN(Date.parse(d.dueDate))) err('invalid due date ' + d.dueDate)
    const hrs = cellText(row, 'estimatedHours')
    if (hrs !== '') { const h = Number(hrs); if (Number.isNaN(h) || h < 0 || h > 999) err('estimatedHours must b
    e 0-999'); else d.estimatedHours = h }
```

```
      const rate = cellText(row, 'hourlyRate')
      if (rate !== '') { const x = Number(rate); if (Number.isNaN(x) || x < 0) err('hourlyRate must be a number >=
       0'); else d.hourlyRate = x }
      if (!errors.some(e => e.row === r)) valid.push(d)
    })

    // 4. ticket numbers and insert
    if (valid.length) {
      const { n } = await SELECT.one.from(Tickets).columns('count(*) as n')
      valid.forEach((d, i) => { d.ticketNumber = 'T-' + String(n + i + 1).padStart(5, '0') })
      await INSERT.into(Tickets).entries(valid)
    }
    return { inserted: valid.length, failed: new Set(errors.map(e => e.row)).size, errors: errors.slice(0, 200) }
  })
```

Notes: the insert goes straight to the database (active records), so service-level validations of draft saves do not run; that is why the checks are repeated in the action. For huge files use batches and a background job.

Story 9.4 - Frontend: button and dialog

Goal: A user-friendly upload dialog.

Your tasks

1. Add a custom action to the List Report toolbar in the manifest (enabled without selection).
2. Write the handler: open a dialog with a file chooser restricted to .xlsx, a short instruction text and a link to the template, and Upload and Cancel buttons (Upload disabled until a file is chosen).
3. Read the file in the browser and send it to the action (research how to read a file as an array buffer and encode it).
4. Handle CSRF token fetching if the app runs behind the approuter later.
5. Show a busy indicator while uploading.

Where to look / topics to explore

- Custom actions in Fiori elements V4 (controlConfiguration actions: press, enabled, requiresSelection)
- sap.ui.unified.FileUploader or the HTML file input
- FileReader / File.arrayBuffer

Acceptance criteria

- The dialog opens, Upload is disabled until a file is chosen, and the request reaches the backend.

Solution

Toolbar action in the manifest (custom action; no selection needed):

manifest.json

```
 "controlConfiguration": { "@com.sap.vocabularies.UI.v1.LineItem": { "actions": {
   "UploadExcel": {
     "press": "desk.tickets.ext.ListActions.onUploadExcel",
     "visible": true, "enabled": true,
     "text": "Upload Excel",
     "requiresSelection": false,
     "position": { "placement": "After", "anchor": "DataFieldForAction::DeskService.EntityContainer::reassignOver
     due" }
   } } } }
```

Handler module (custom action handler file):

webapp/ext/ListActions.js

```
 sap.ui.define([
   "sap/m/Dialog", "sap/m/Button", "sap/m/Text", "sap/m/Link", "sap/m/VBox", "sap/m/MessageBox",
   "sap/m/MessageToast", "sap/ui/unified/FileUploader"
 ], function (Dialog, Button, Text, Link, VBox, MessageBox, MessageToast, FileUploader) {
   "use strict";
   const BASE = "/odata/v4/desk/";

   function toBase64(buffer) {                    // chunked: avoids stack overflow on big files
     const bytes = new Uint8Array(buffer); let s = "";
     for (let i = 0; i < bytes.length; i += 0x8000) s += String.fromCharCode.apply(null, bytes.subarray(i, i + 0x
     8000));
     return btoa(s);
```

```
  }
  async function csrfToken() {                    // needed behind the approuter, harmless locally
    const r = await fetch(BASE, { method: "HEAD", headers: { "X-CSRF-Token": "Fetch" } });
    return r.headers.get("x-csrf-token") || "";
  }
  async function send(file) {
    const content = toBase64(await file.arrayBuffer());
    const res = await fetch(BASE + "uploadExcel", {
      method: "POST",
      headers: { "Content-Type": "application/json", "X-CSRF-Token": await csrfToken() },
      body: JSON.stringify({ fileName: file.name, content })
    });
    const json = await res.json();
    if (!res.ok) throw new Error(json.error?.message || res.statusText);
    return json;
  }

  return {
    // 'this' differs between UI5 versions: log it once. In recent versions it gives access to the extension API
     (refresh()).
    onUploadExcel: function () {
      const ext = this;
      let file = null;
      const uploader = new FileUploader({
        width: "100%", fileType: ["xlsx"], sameFilenameAllowed: true, placeholder: "Choose an .xlsx file",
        change: e => { file = e.getParameter("files")[0]; dialog.getBeginButton().setEnabled(!!file); }
      });
      const dialog = new Dialog({
        title: "Upload tickets from Excel",
        content: new VBox({ items: [
          new Text({ text: "Use the template columns: title, category, priority, customerEmail, ..." }),
          new Link({ text: "Download template", href: "templates/Ticket_Upload_Template.xlsx", target: "_blank"
    }),
          uploader ] }).addStyleClass("sapUiSmallMargin"),
        beginButton: new Button({ text: "Upload", type: "Emphasized", enabled: false, press: async () => {
          dialog.setBusy(true);
          try {
            const result = await send(file);
            dialog.close();
            if (ext && ext.refresh) ext.refresh();           // reload the table
            const lines = (result.errors || []).slice(0, 20).map(e => "Row " + e.row + ": " + e.message);
            const more = result.failed > 20 ? "\n... and " + (result.failed - 20) + " more rows" : "";
            const text = result.inserted + " tickets created, " + result.failed + " rows rejected.";
            if (lines.length) MessageBox.warning(text + "\n\n" + lines.join("\n") + more);
            else MessageToast.show(text);
          } catch (err) { dialog.setBusy(false); MessageBox.error(err.message); }
        } }),
        endButton: new Button({ text: "Cancel", press: () => dialog.close() }),
        afterClose: () => dialog.destroy()
      });
      dialog.open();
    }
  };
});
```

Place the template under `webapp/templates/`. If `this` has no `refresh`, add a List Report controller extension (page map in Fiori tools) and call `this.base.getExtensionAPI().refresh()` there, or call `refresh()` on the table binding. Registering the module path as `desk.tickets.ext` follows from the app id.

Story 9.5 - Result feedback and refresh

Goal: Users understand what happened.

Your tasks

1. Show a success message with counts. If errors exist show a dialog listing row numbers and messages (limit to the first 20 and say how many more).
2. Refresh the list so new tickets appear without reloading the page.
3. Offer download of the error list (research a simple way; optional).
4. Test with all three files and write what a user sees.

Where to look / topics to explore

- sap.m.MessageBox, MessageToast
- ExtensionAPI refresh of the list report

Acceptance criteria

- New tickets show up after upload, errors are readable.

Solution

The handler above already shows toast/dialog and refreshes. Improvements: build the error list as a table inside a `sap.m.Dialog` (columns Row and Message); offer "Download errors" by creating a CSV Blob from `result.errors` and a temporary link; keep the first 20 visible and mention the remainder. Test matrix: valid file -> toast only; mixed file -> warning dialog with row numbers; broken file -> error dialog "Missing columns: ...".

Story 9.6 - Approach A: spreadsheet importer component

Goal: Compare with a ready-made solution.

Your tasks

1. In a copy of the project add the spreadsheet importer according to its documentation (component registration, custom action, optional configuration).
2. Map the template columns to entity properties and test with your valid file.
3. Check how it handles draft-enabled entities, value help fields (codes versus text) and errors.
4. Update your comparison table with real experience, including the licence consequences for production use.

Where to look / topics to explore

- docs.spreadsheet-importer.com (Getting Started for Fiori elements)
- Live demo

Acceptance criteria

- The valid file uploads with approach A.
- Your comparison table contains real findings.

Solution

Follow the current "Getting started" page of the importer documentation (docs.spreadsheet-importer.com): install the component, register it (component usage) in the manifest, add a custom toolbar action whose handler opens the upload dialog, and configure the column mapping/options. Component names and versions change between releases, so copy the exact names from the documentation instead of from memory. Findings to record: how value-help fields are mapped (codes versus texts), how errors are shown, whether upload creates drafts or active rows, and the licence terms for production.

Story 9.7 - Edge cases, security and tests

Goal: Make it robust.

Your tasks

1. Check the maximum request body size of the server and configure it for your upload limit.
2. Reject files above your limit with a friendly message on the client and the server.
3. Check formula cells, dates stored as text, merged cells, trailing blank rows, leading/trailing spaces in codes.
4. Make sure only authorised roles can upload (preparation for security; write the annotation you would use).
5. Write automated tests for the action using small generated workbooks.

Where to look / topics to explore

- Server body parser limit configuration
- Library API for cell values and dates

Acceptance criteria

- Oversize and malformed files are handled; tests exist.

Solution

Body size: CAP uses an Express body parser with a default limit; raise it for your upload limit (base64 adds about 33%). Check the current setting name in capire (cds.server.body_parser.limit):

package.json

```
"cds": { "server": { "body_parser": { "limit": "15mb" } } }
```

Authorisation preparation (Milestone for security): restrict the action to a role.

srv/authorization.cds

```
annotate DeskService.uploadExcel with @(requires : 'TicketAdmin');
```

Test idea with generated workbooks:

test/upload.test.js

```
const ExcelJS = require('exceljs')
async function workbook(rows) {
  const wb = new ExcelJS.Workbook(); const ws = wb.addWorksheet('Tickets')
  ws.addRow(['title', 'category', 'priority', 'customerEmail', 'dueDate'])
  rows.forEach(r => ws.addRow(r))
  return (await wb.xlsx.writeBuffer()).toString('base64')
}
// it('rejects unknown category', async () => {
//   const res = await POST('/odata/v4/desk/uploadExcel', { fileName: 'a.xlsx',
//     content: await workbook([['Printer', 'XX', 'M', 'anna.becker@example.com', '2026-12-01']]) })
//   expect(res.data.failed).toBe(1)
// })
```

Milestone checkpoint

Question	Answer
Why parse on the server and not in the browser?	Validation and data access (lookups by e-mail) belong with the data; the browser only transports the file.
Why return row numbers in errors?	Users fix the right row in their sheet.

Milestone 10 - Review, Polish and Coverage Check

Item	Detail
Mini project	Walk through the finished app with a checklist and fill the annotation coverage matrix.
Level / effort	Intermediate / 3-5 hours
Annotations practised	Quality review, accessibility, translations, documentation.

Learning outcomes

- Prove every annotation topic was used.
- Document the app for the next learner.

Story 10.1 - Annotation coverage matrix

Goal: Evidence that you practised everything.

Your tasks

1. For each row of the coverage matrix at the start of this document, write the file and line where you used it and a screenshot of the effect.
2. Mark any row you skipped and plan a follow-up.

Acceptance criteria

- The matrix is complete.

Solution

Fill the matrix: annotation, file, line, screenshot. Compare with the coverage table at the start; every row must have evidence.

Story 10.2 - Accessibility, texts and translations

Goal: Quality.

Your tasks

1. Check that every label comes from i18n and translate the app into a second language.
2. Navigate the list report with keyboard only and note problems.
3. Check colour is never the only carrier of meaning (criticality has icons or texts).

Acceptance criteria

- App works in two languages and with keyboard.

Solution

Create a second i18n file (for example i18n_de.properties), switch the browser language and verify every label. Keyboard test: Tab through filter bar, table, toolbar; check focus visibility and that dialogs can be closed with Escape.

Story 10.3 - Document and explain

Goal: Teach it back.

Your tasks

1. Write a README: architecture, how to run, annotation file map, manifest settings, known limitations.
2. Explain the project in 10 minutes to someone; collect questions you could not answer and study them.

Acceptance criteria

- README exists and you can explain each annotation file.

Solution

Suggested README outline: 1 Purpose, 2 Architecture picture, 3 Run locally, 4 Annotation file map (file -> topic), 5 Manifest settings, 6 Backend actions, 7 Known limitations, 8 Learning links.

Milestone checkpoint

Question	Answer
Which three annotations gave you the biggest surprise and why?	Personal reflection; compare with the feature showcase.